CollabBoard

ADIM ADIM REHBER

GitHub'dan canlıya

Bir değişiklik bilgisayardan siteye nasıl gider? Dal, commit, pull request, CI, merge ve Railway — ne olduklarını, ekranda nasıl göründüklerini ve neden var olduklarını, bizim projemizin gerçek geçmişi üzerinden.

commit → push → dal → pull request → CI ✓ → merge → main → Railway → Online

Bu rehberi nasıl kullanırsın?

- Kavramları oturtmak için baştan oku: **Bölüm 1–3**.
- GitHub ekranlarında neyin ne olduğunu görmek için: **Bölüm 4** (gerçek ekran görüntüleri, numaralı işaretlerle).
- Önünde birleştirilecek bir PR varsa: **Bölüm 5** ve en sondaki **10 adımlık kopya kâğıdı**.
- Bir şey beklediğin gibi gitmediyse: **Bölüm 7 — Sık karşılaşılan durumlar**.

Erdem için hazırlandı · 23 Ekim 2026 · Depo: `github.com/erdemsidal/collab-board` · Bu tarihte `main` = `0db46b9` (PR #6). Tüm saatler Türkiye saatidir (UTC+3).

İÇİNDEKİLER

Bu rehberde

1 **Büyük resim** — kod nereden nereye gider, kim neyi yapar 3

2 **Temel kavramlar** — depo, dal, commit, push, pull, PR, CI, merge, conflict 5

3 **CI'ımız** — GitHub Actions ne çalıştırıyor, "Build skipped" neden normal 8

4 **GitHub sayfalarını okumak** — gerçek ekran görüntüleri, işaretli 10

5 **Bir değişikliği canlıya almak** — bugün kullandığımız akış, tıklama tıklama 17

6 **Gerçek geçmişimiz** — PR #1'den #6'ya, yaşanan sorunlar ve sebepleri 22

7 **Sık karşılaşılan durumlar** — belirti → sebep → ne yapmalı 26

8 **Sözlük** — terimler, kısa tanımlarıyla 28

9 **Kopya kâğıdı** — değişikliği canlıya alma, 10 adım 30

Tek cümlelik özet

`main` dalı = canlı site. Kod `main`'e ancak bir pull request üzerinden, testler (CI) yeşilken ve sen **Merge pull request** → **Confirm merge** dediğinde girer; girdiği an Railway onu kendiliğinden yayına alır.

Bu rehberde kullanılan işaretler

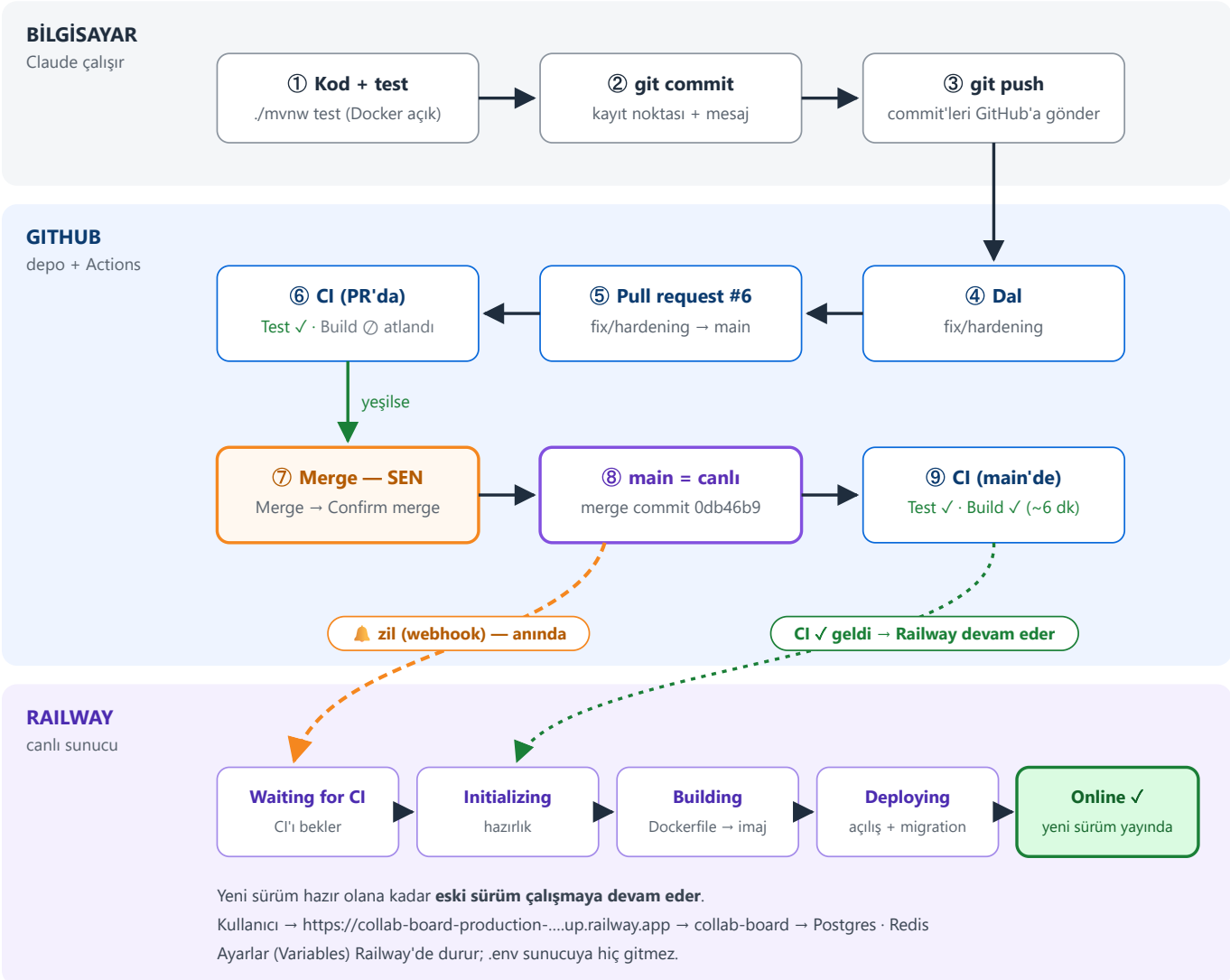
Gördüğün	Anlamı
Merge pull request	Ekranda tıklayacağın bir düğme ya da menü öğesinin **ekrandaki adı**. GitHub ve Railway arayüzü İngilizce olduğu için adlar İngilizce bırakıldı.
`git push`, `main`	Komut, dal adı, dosya adı ya da kod parçası.
`Ctrl` + `F5`	Klavyede basılacak tuşlar.
Turuncu çerçeve + ❶	Ekran görüntüsündeki işaret; altındaki listede aynı numaranın açıklaması var.
TEMSİLİ ÇİZİM	Ekran görüntüsü alınamayan yerler (giriş gerektiren GitHub kutuları, Railway paneli) için çizilmiş benzeri. Yerleşim birebir olmayabilir; düğme ve başlık yazıları gerçek ekrandakilerdir.
✓ yeşil · ✗ kırmızı · ● sarı · ⊘ gri	Kontrol sonucu: geçti · başarısız · hâlâ çalışıyor · atlandı (skipped).

Bölüm 1 · BÜYÜK RESİM

Kod nereden nereye gider?

Bir satır kodun siteye ulaşması için üç yer var. Her biri tek bir şeyden sorumlu; ne olduğunu bilmek, bir şey takıldığında *nereye bakacağını* bilmek demek.

Yer	Ne tutar?	Kim, ne yapar?
Bilgisayar (yerel depo)	Kodun çalışma kopyası. Ayrıca `.env` dosyası: yerel şifreler ve anahtarlar — **asla GitHub'a gitmez**.	Claude değişikliği burada yazar, testleri çalıştırır, commit'ler ve GitHub'a gönderir (push).
GitHub `erdemsidal/collab-board`	Kodun "resmî" kopyası: dallar, commit geçmişi, pull request'ler ve CI sonuçları. Depo herkese açık (Public).	GitHub Actions her PR'da testleri çalıştırır (CI). **Sen** PR'ı birleştirirsin (merge).
Railway proje `collab-board`	Çalışan uygulama (`collab-board`), veritabanı (`Postgres`) ve `Redis`. Canlı ayarlar `Variables` bölümünde.	`main`'e yeni kod girince kendiliğinden derler ve yayına alır. Sen yalnızca durumunu izlersin.



Yeni sürüm hazır olana kadar **eski sürüm çalışmaya devam eder**.
Kullanıcı → https://collab-board-production-....up.railway.app → collab-board → Postgres · Redis
Ayarlar (Variables) Railway'de durur; .env sunucuya hiç gitmez.

main = canlı

`main`, GitHub'daki dallardan yalnızca biri — ama Railway yalnızca onu dinliyor. `main`'e giren her commit birkaç dakika içinde siteye çıkar. Bu yüzden `main`'e doğrudan yazılmaz: her değişiklik bir dalda hazırlanır, PR'da test edilir ve **sen** birleştirdiğinde girer. **Confirm merge** düğmesi, pratikte **"yayınla"** düğmesidir.

Kim, neyi yapar?

Adım	Kim	Ne olur	Ayrıntı
A	Claude	Dal açar, kodu ve testleri yazar, commit'ler, push eder, PR açar. CI kendiliğinden başlar.	Bölüm 5 / A
B	**Sen**	PR'ı açar, kontrollerin yeşil olduğunu görür, **Merge pull request** → **Confirm merge**.	Bölüm 5 / B
C	Otomatik	GitHub, Railway'e haber verir; Railway CI'ı bekler, derler, yayına alır.	Bölüm 5 / C
D	**Sen**	Railway'de yeni sürümün **ACTIVE** olduğunu ve logları kontrol eder, siteyi dener.	Bölüm 5 / D

Gerçek bir örnek: PR #6'nın yolculuğu (23 Ekim 2026)

Aşağıdaki saatler GitHub'daki kayıtlardan (commit'ler, Actions çalışmaları ve Railway'in GitHub'a yazdığı yayın kayıtları) alındı.

- 19:45 · **Commit + push.** İki commit (`055c8dc`, `173f7fe`) `fix/hardening` dalında hazırlandı ve GitHub'a gönderildi.
- 19:46 · **PR #6 açıldı.** "Sağlamlaştırma: olmayan adres 404, canlıda metrikler kapalı, testler .env'den yalıtıldı".
- 19:46 → 19:47 · **CI (PR'da) ✓** Actions çalışması #41: Test 1 dk 40 sn'de geçti, Build atlandı. Toplam 1 dk 44 sn.
- 20:39 · **Sen birleştirdin.** Merge commit `0db46b9` `main`'e girdi.
- 20:39 · **Zil çaldı.** Railway yeni bir yayın açtı ve GitHub'a "başladı" diye yazdı (20:39:42) — ekranda **Waiting for CI**.
- 20:39 → 20:45 · **CI (main'de) ✓** Actions çalışması #42: Test 2 dk 44 sn, ardından Build 2 dk 45 sn. Toplam 5 dk 40 sn.
- 20:46 · **Canlıda.** Railway yeni sürümü yayına aldı (GitHub'daki yayın kaydı: 20:46:51 "success").

Merge'den canlıya ≈ 7 dakika. Bunun ~6 dakikası `main`'deki CI'ı beklemekle geçiyor; bu bilinçli bir tercih (Bölüm 5 / C: "Wait for CI").

Bölüm 2 · TEMEL KAVRAMLAR

On iki kavram, her biri üç satırda

Her kavram için önce akılda kalsın diye bir **benzetme**, sonra **gerçekte** tam olarak ne olduğu, en sonda da **bizde** nerede gördüğün. Benzetme yalnızca yardımcıdır; işi yapan tanımdır.

Repository — depo	repo
Benzetme	Projenin bütün dosyalarıyla birlikte, her değişikliğin yazıldığı bir "defter" de içeren klasör.
Gerçekte	Dosyalar + gizli `.git` klasöründeki bütün commit geçmişi. Aynı deponun birden fazla kopyası olur: bilgisayardaki **yerel** kopya ve GitHub'daki **uzak** kopya (Git'te adı `origin`). İkisi kendiliğinden eşitlenmez; push ve pull ile eşitlenir.
Bizde	`github.com/erdemsidal/collab-board` — **Public** (herkes görebilir, yalnızca sen değiştirebilirsin). 23 Ekim'de `main`'de 57 commit, depoda 7 dal var.

Commit	kayıt noktası
Benzetme	Oyundaki "kaydet" — ama her kayda neden kaydettiğini anlatan bir not yazıyorsun.
Gerçekte	Belirli bir andaki dosyaların tam fotoğrafı + yazar + tarih + mesaj + **bir önceki commit'e (ebeveyn) bağlantı**. Kimliği 40 haneli bir özettir (hash); genelde ilk 7 hanesi kullanılır: `055c8dc`. Commit yalnızca bilgisayarda oluşur; GitHub'a gitmesi için push gerekir.
Bizde	Mesajlar İngilizce ve "ne"den çok "neden"i anlatır (ör. *Keep the developer's .env out of the tests*). Sondaki `Co-Authored-By: Claude …` satırı yüzünden GitHub commit'lerin yanında iki avatar gösterir: sen ve Claude.

Branch — dal	
Benzetme	Ana metnin bir kopyası üzerinde çalışmak; beğenince düzeltmeleri ana metne işlersin, beğenmezsen kopyayı atarsın.
Gerçekte	Bir commit'i gösteren, adı olan bir **etiket**. O dalda yeni commit atılınca etiket kendiliğinden yeni commit'e kayar. `main` de böyle bir etikettir; tek farkı Railway'in onu dinlemesi.
Bizde	`main` ve birleştirilmiş 6 dal. Ad düzeni: `feature/…` yeni özellik, `fix/…` düzeltme, `docs/…` belge (ör. `fix/hardening`, `feature/brevo-mail`).

Push	gönder
Benzetme	Defterdeki yeni sayfaların fotokopisini merkeze yollamak.
Gerçekte	Yerel commit'leri GitHub'daki aynı adlı dala göndermek: `git push -u origin fix/hardening`. `-u` yalnızca ilk seferde gerekir: "bu yerel dal, GitHub'daki şu dalla eşleşsin" der.
Bizde	Push'u Claude yapar. Açık bir PR'ın dalına push edilirse PR kendiliğinden güncellenir ve CI yeniden çalışır.

Pull çek	
Benzetme	Merkezdeki yeni sayfaları kendi defterine eklemek.
Gerçekte	GitHub'daki yeni commit'leri bilgisayara almak: `git pull` (= önce indir [`fetch`], sonra yerel dala kat [`merge`]). Ör. bir PR birleştikten sonra yerel `main`'i güncellemek için: `git checkout main` ve `git pull`.
Bizde	"Pull request" ile karıştırma. Oradaki *pull*, "benim dalımdaki değişiklikleri al (çek)" isteğidir.

Pull request — PR birleştirme isteği	
Benzetme	"Şu düzeltmeleri ana metne işleyelim mi?" diye açılmış bir dosya: içinde gerekçe, tartışma, test sonuçları ve farkın kendisi var.
Gerçekte	İki dal arasındaki farkı gösteren GitHub sayfası. Yön her zaman **kaynak dal** (head, ör. `fix/hardening`) → **hedef dal** (base, ör. `main`). Bir numarası (#6) ve dört sekmesi vardır: Conversation, Commits, Checks, Files changed. PR açık kaldıkça dala eklenen her commit PR'a da eklenir.
Bizde	PR'ları Claude, senin GitHub hesabınla açar (bu yüzden listede hep "by erdemsidal" yazar). Açıklama her zaman **Neden / Ne değişti / Test / Not** başlıklarıyla yazılır; **Not**, birleştirmeden önce senin yapman gerekenleri söyler.

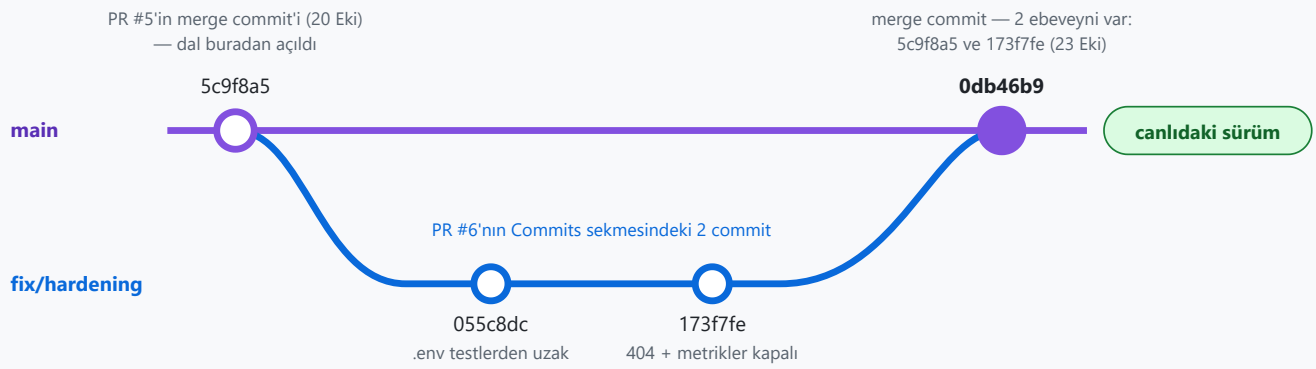
CI — Continuous Integration sürekli entegrasyon	
Benzetme	Fabrikada her parçanın banda girmeden önce geçtiği kalite kontrol istasyonu.
Gerçekte	Her PR'da ve `main`'e her girişte, kodun GitHub'ın kendi temiz makinesinde (**GitHub Actions**) derlenip bütün testlerin çalıştırılması. Tarif `.github/workflows/ci.yml` dosyasında. "Benim bilgisayarımda çalışıyordu" sorununu yakalar: temiz makinede senin `.env`'in, önbelleğin, kurulu programların yoktur.
Bizde	Adı **CI/CD Pipeline**; iki işi var: 🧪 **Test** ve 🏗️ **Build**. Ayrıntısı Bölüm 3'te.

Checks — kontroller ✓ ✗ ● ⊘	
Benzetme	Kalite kontrolden geçen parçanın üstüne basılan damga.
Gerçekte	CI'daki her işin sonucu, üzerinde çalıştığı commit'in yanına işaret olarak yazılır: ✓ **yeşil** geçti, ✗ **kırmızı** başarısız, ● **sarı** hâlâ çalışıyor, ⊘ **gri** atlandı (skipped). Bir işin atlanması hata değildir.
Bizde	PR'da Test ✓ + Build ⊘ (atlandı) görürsün ve bu **normaldir**; `main`'de ikisi de ✓ olmalı.

Merge birleştirme	
Benzetme	Kopyadaki düzeltmeleri ana metne işlemek.
Gerçekte	Bir dalın commit'lerini başka bir dala katmak. PR sayfasındaki Merge pull request düğmesi, kaynak daldaki commit'leri hedef dala (`main`) katar. Sonrasında PR kapanır ve mor Merged olur.
Bizde	Merge'ü **yalnızca sen** yaparsın; bu, yayın kararıdır.

Merge commit birleştirme kaydı

Benzetme	Ana metne "şu tarihte şu düzeltme dosyası işlendi" diye düşülen not.
Gerçekte	Birleştirmeyi kayda geçiren özel commit. Normal commit'in tek ebeveyni vardır; merge commit'in **iki**: hedef dalın o anki ucu ve kaynak dalın ucu. Varsayılan mesajı: `Merge pull request #6 from erdemsidal/fix/hardening`.
Bizde	Canlıya çıkan sürüm tam olarak merge commit'tir: bugün canlıda `0db46b9` çalışıyor. Aşağıdaki çizimde gerçek hâli var.



"Delete branch" dalı sil

Benzetme	İşlenmiş düzeltme kopyasını rafa kaldırmak.
Gerçekte	Merge'den sonra PR sayfasında çıkan düğme; dalı **GitHub'dan** siler. Commit'ler kaybolmaz, çünkü zaten `main`'in içindedirler. Aynı yerde **Restore branch** ile geri getirilebilir. Silmek zorunlu değil.
Bizde	Şimdiye kadar hiçbir dal silinmedi: depoda `main` + birleştirilmiş 6 dal duruyor. Zararı yok; yalnızca dal listesi kalabalık. Depo ayarında "birleşince dalı otomatik sil" kapalı.

Conflict — çakışma

Benzetme	İki kişi aynı paragrafı farklı biçimde düzeltmiş; hangisinin kalacağına makine karar veremez.
Gerçekte	Dal açıldıktan sonra `main`'de de değişen **aynı dosyanın aynı satırları** dalda da değiştiyse Git ikisini kendiliğinden birleştiremez. PR'da **This branch has conflicts that must be resolved** yazar ve birleştirme düğmesi kullanılamaz. Çözümü kodu bilen yapar: dal güncel `main`'le birleştirilir, çakışan satırlar elle düzeltilir, yeniden push edilir.
Bizde	Henüz hiç yaşanmadı. Görürsen birleştirme; Claude'a "PR #N'de conflict var" demen yeter.

Bölüm 3 · CI'IMIZ

GitHub Actions ne çalıştırıyor?

CI'ın bütün tarifi tek bir dosyada: `.github/workflows/ci.yml`. Bu bölüm o dosyayı satır satır değil, **ne zaman**, **ne** ve **neden** sorularıyla anlatıyor.

Ne zaman çalışır? (tetikleyiciler)

```
name: CI/CD Pipeline

on:
  push:
    branches: [main, develop]     # ① bu dallara kod girince
  pull_request:
    branches: [main]              # ② hedefi main olan bir PR'da
```

#	Olay	Bizde ne demek?
①	**push** — `main`'e (ya da `develop`'a) commit girdi	Merge de bir push'tur: merge commit `main`'e yazılır, CI çalışır (Actions'ta "pushed by"). `develop` diye bir dalımız yok; satır boilerplate'ten kaldı, zararsız.
②	**pull_request** — hedefi `main` olan bir PR	GitHub'ın varsayılanıyla üç durumda çalışır: PR **açıldığında** (*opened*), dalına **yeni commit geldiğinde** (*synchronize*) ve kapatılıp **yeniden açıldığında** (*reopened*). Başlığı ya da hedef dalı değiştirmek (*edited*) CI'ı **çalıştırmaz** — PR #5'te bunu yaşadık (Bölüm 6).

> **Dikkat: hedefi main olmayan PR'da CI hiç çalışmaz**
> `branches: [main]` filtresi yüzünden, örneğin hedefi `feature/invitations-profile` olan bir PR'da hiçbir test koşmaz. Ayrıca bir dala push etmek, o dal için açık bir PR yoksa CI'ı tetiklemez.

Ne yapar? (iki iş)

İş	Ne zaman	Ne yapar	PR #6'da süre
🧪 **Test**	Her tetiklemede	Kodu indirir (*Checkout*), Java 21 (Temurin) kurar, `mvn verify` ile derleyip **bütün testleri** çalıştırır. Sonuç raporlarını `test-reports` adıyla saklar (çalışma sayfasında *Artifacts*).	PR: 1 dk 40 sn main: 2 dk 44 sn
🏗️ **Build**	Yalnızca **push**'ta ve Test geçtiyse	`mvn package -DskipTests` ile JAR üretir. `main`'deyse ayrıca Docker imajını derleyip GitHub'ın imaj deposuna (`ghcr.io`) `latest` ve commit kimliği etiketleriyle gönderir.	PR: atlandı (0 sn) main: 2 dk 45 sn

```
  build:
    name: 🏗️ Build
    needs: test                                  # Test geçmeden başlamaz
    if: github.event_name == 'push'              # PR'da (pull_request) atlanır
```

"Build skipped" neden normal?

PR aşamasında sorulan tek soru şu: "*Bu kod main'e girerse bir şey bozulur mu?*" Bu sorunun cevabını **Test** verir. Build'in işi paketlemek ve imajı yayınlamak; henüz `main`'e girmemiş, belki hiç girmeyecek bir kodun imajını üretmek boşa iş. Bu yüzden `if: github.event_name == 'push'` satırı Build'i PR'da bilerek atlatır.

- Atlanan iş **hata sayılmaz**: PR'da yine `All checks have passed` görürsün.
- Commits sekmesinde son commit'in yanındaki **✓ 1 / 2** bundan: iki kontrolden biri (Test) geçti, diğeri (Build) atlandı.
- Merge'den sonra, `main`'deki çalışmada Build **çalışır** ve ✓ olmalıdır. Railway de iki kontrolü birden bekler.

> **Railway bu imajı kullanıyor mu?**
> Hayır. Railway kodu GitHub'dan alıp `Dockerfile` ile **kendisi** derliyor (Bölüm 5 / C, "Building"). `ghcr.io`'daki imaj, "bu Dockerfile temiz bir makinede de derleniyor mu?" sorusunun cevabı ve ileride başka bir yere taşınmak istersen hazır bir paket. CI'da bilerek bir **deploy** işi yok: eskiden yalnızca ekrana yazı yazıp yeşil yanan sahte bir deploy adımı vardı, hiçbir şey yayınlanmadığı hâlde "başarılı" görünüyordu; kaldırıldı (commit `8e4bc0f`, PR #1).

Testcontainers: testler gerçek veritabanıyla koşar

Testler sahte (mock) bir veritabanı kullanmaz. **Testcontainers** kütüphanesi testler başlarken Docker'da gerçek bir **PostgreSQL** ve **Redis** konteyneri açar, adreslerini çalışma anında Spring'e verir (`IntegrationTestBase`) ve testler bitince kapatır. Böylece Flyway migration'ları, JPA eşlemeleri ve gerçek STOMP trafiği de denenmiş olur.

- **CI'da:** GitHub'ın Ubuntu makinesinde Docker hazır; `ci.yml`'de ayrıca veritabanı servisi tanımlı değil, gerek yok.
- **Yerelde:** testleri (`./mvnw test`) çalıştırmadan önce **Docker Desktop açık olmalı**; yoksa testler konteyner açamaz.
- 23 Ekim itibarıyla 96 test var: 92 entegrasyon + 4 birim (PR #6).

Kırmızı geçen 30 çalışma

27 Temmuz – 3 Ağustos arasında kod doğrudan `main`'e yazılıyordu ve Actions'taki **ilk 30 çalışmanın hepsi kırmızıydı** (#1–#30). Sebep testler değildi: `ci.yml` içindeki `JWT_SECRET` geçerli bir Base64 değeri değildi, uygulama tek test koşmadan açılamıyordu. PR #1 bunu düzeltti (commit `8e4bc0f`); o günden beri 12 çalışmanın 12'si yeşil (#31–#42).

> **Ders**
> Hep kırmızı yanan bir ışık, bir süre sonra kimsenin bakmadığı bir ışıktır. CI'ın değeri, kırmızının **istisna** olmasından gelir. Bugün kırmızı bir *X* gördüğünde bu gerçekten "bir şey bozuldu" demek.

Sarı uyarılar (Annotations)

Çalışma sayfalarının altında sarı üçgenli **Annotations** görürsün: "Node.js 20 is deprecated…", "setup-java v4 is deprecated…", "The ubuntu-latest label will migrate to Ubuntu 26…". Bunlar GitHub'ın **bakım uyarıları**; testleri etkilemez ve çalışmayı kırmızı yapmaz. İleride `ci.yml`'deki eylem sürümlerini (ör. `actions/setup-java@v5`) güncelleyen küçük bir PR ile kapanırlar.

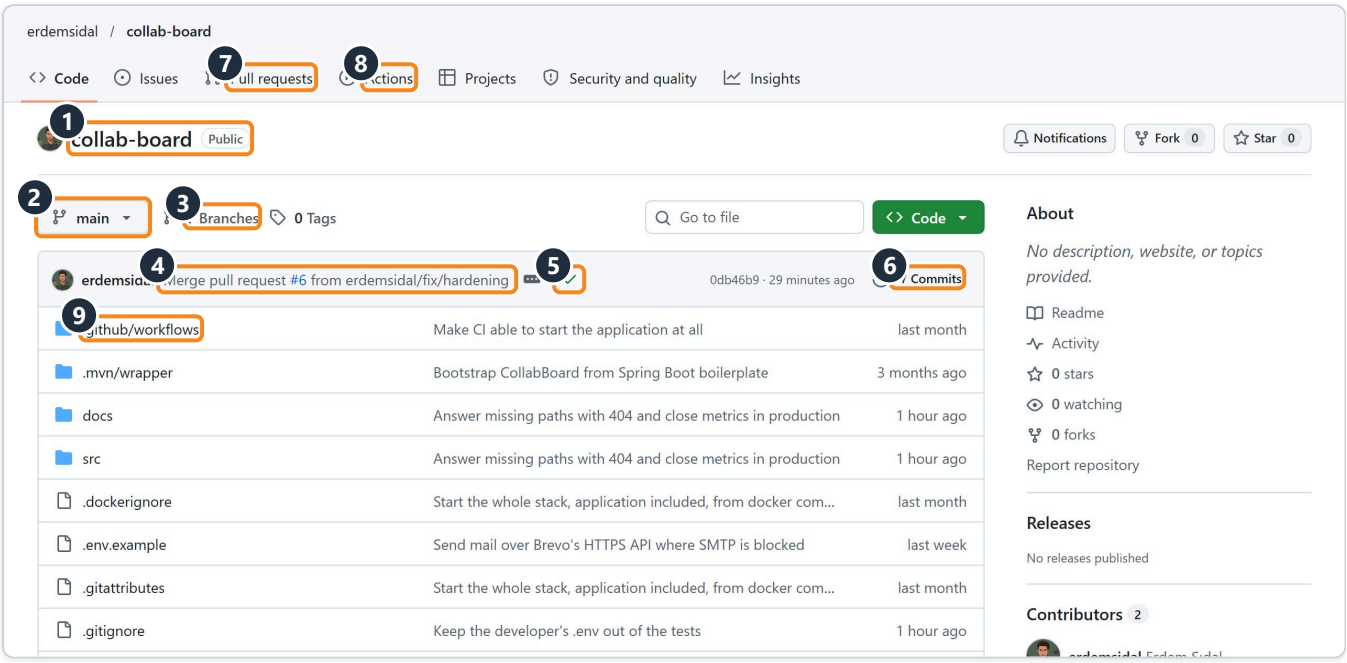
Bölüm 4 · GITHUB SAYFALARINI OKUMAK

Hangi ekranda ne var?

Ekran görüntüleri 23 Ekim 2026 akşamı, deponun herkese açık hâlinden, **giriş yapmadan** alındı. Sen giriş yapmış olduğun için birkaç fark görürsün: sağ üstte **Sign in** yerine profil fotoğrafın, PR'larda yorum kutusu, açık PR'larda merge kutusu (Bölüm 5). Turuncu çerçeveleri ve numaraları ben ekledim.

4.1 Depo ana sayfası

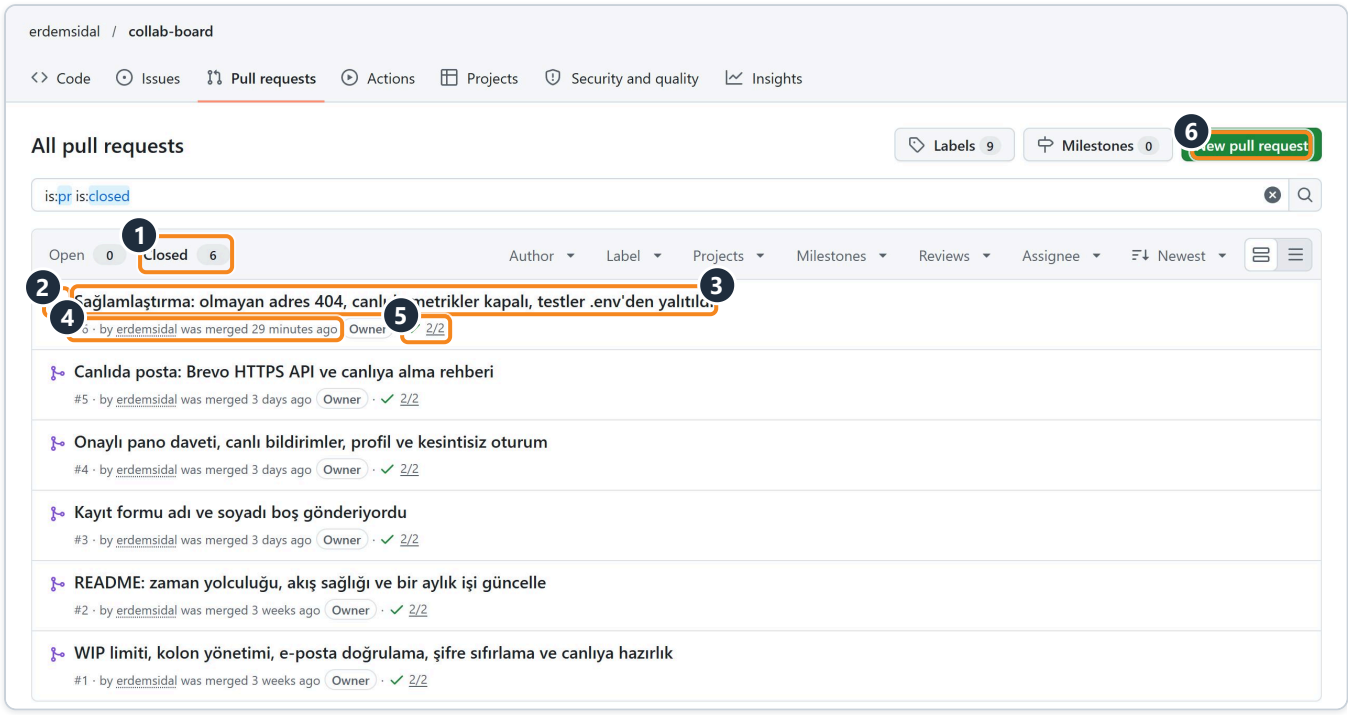
Adres: github.com/erdemsidal/collab-board — üstteki sekmelerden **Code**.



1. **Depo adı ve "Public".** Herkes kodu ve geçmişi görebilir; değiştirme ve birleştirme yetkisi yalnızca sende. Şifreler bu yüzden asla depoya girmez.
2. **Dal seçici.** Şu an `main`'in içeriğini görüyorsun. Tıklayınca dallar listelenir; birini seçince sayfa o dalın dosyalarını gösterir.
3. **7 Branches.** Bütün dallar: `main` + birleştirilmiş 6 dal (hiçbiri silinmedi).
4. **Seçili dalın en son commit'i.** `main`'deyken bu satır = **canlıdaki sürüm**. Burada PR #6'nın merge commit'i (`0db46b9`).
5. **CI sonucu.** Yeşil ✓: bu commit için Test ve Build geçti. Kırmızı ✗ olsaydı "main'de bir şey bozuk" demekti. Üzerine tıklayınca hangi kontrollerin çalıştığı açılır.
6. **57 Commits.** `main`'in bütün geçmişi; tıklayınca tarih sırasıyla commit listesi açılır.
7. **Pull requests sekmesi** — açık ve kapanmış PR'lar (4.2).
8. **Actions sekmesi** — CI çalışmalarının listesi (4.7).
9. **.github/workflows** — CI'ın tarifi `ci.yml` bu klasörde.

4.2 Pull requests sekmesi

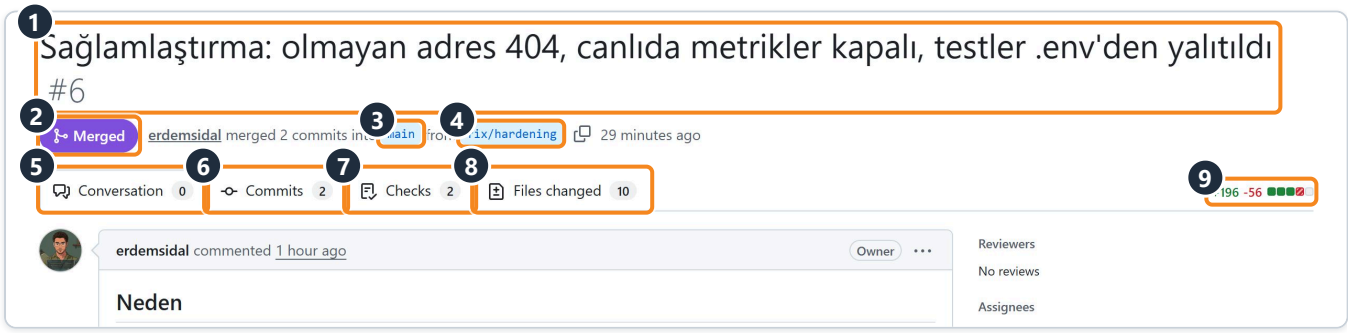
Üstteki **Pull requests** sekmesi. Varsayılan olarak **açık** PR'lar listelenir; aşağıdaki görüntüde **Closed** 'a tıklanmış hâli var.



1. **Open 0 / Closed 6.** Bekleyen PR yok; 6 PR'ın hepsi kapandı. Senin onayını bekleyen bir PR olduğunda **Open** 'da görünür — ilk bakacağın yer orası.
2. **Durum simgesi.** Mor = birleştirildi (merged). Yeşil = açık. Kırmızı = birleştirilmeden kapatıldı. Gri = taslak (draft).
3. **Başlık.** Tıklayınca PR sayfası açılır (4.3).
4. **#6 · by erdemsidal was merged …** PR numarası, açan hesap, birleştirilme zamanı.
5. **✓ 2/2** — kontrollerin özeti; tıklayınca ayrıntısı açılır.
6. **New pull request** — elle PR açmak için. Bizde PR'ları Claude komut satırından açtığı için bu düğmeye gerek kalmıyor.

4.3 PR sayfası — üst kısım

PR #6, Conversation sekmesi.

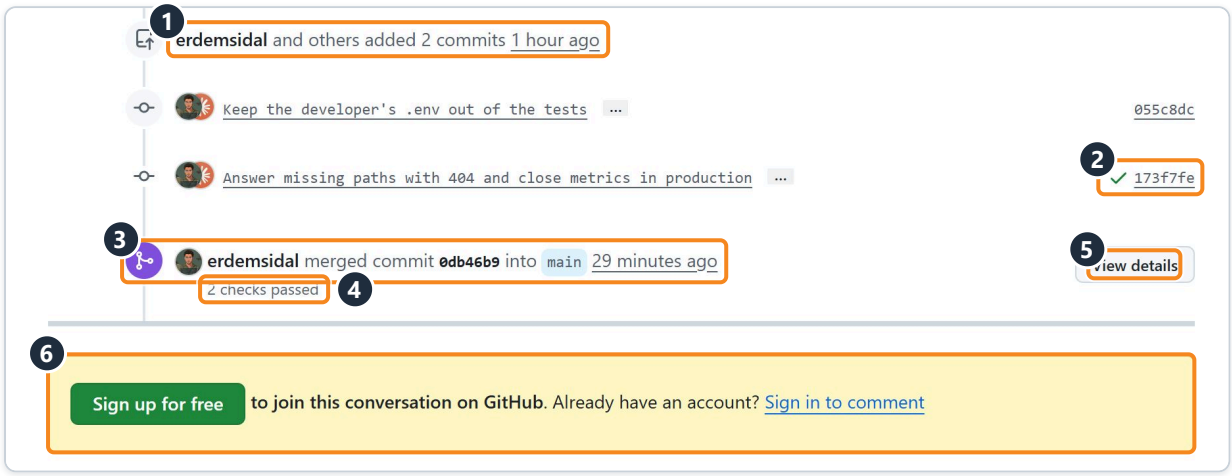


1. **Başlık ve numara.** Başlığı Claude yazar: değişikliğin özeti, Türkçe.
2. **Durum rozeti.** Merged birleştirildi · Open açık, karar bekliyor · Closed birleştirilmeden kapatıldı · Draft taslak, henüz hazır değil.
3. **Hedef dal (base):** `main` — değişikliklerin gireceği yer.
4. **Kaynak dal (head):** `fix/hardening` — değişikliklerin durduğu yer. Cümle şöyle okunur: "*erdemsidal, fix/hardening'den 2 commit'i main'e birleştirdi.*" Açık bir PR'da "wants to merge 2 commits into main from fix/hardening" yazar.
5. **Conversation** — açıklama, yorumlar, olaylar ve en altta merge kutusu.

6. **Commits** — bu dalda hangi commit'ler var (4.5).
7. **Checks** — CI sonuçlarının ayrıntısı.
8. **Files changed** — satır satır fark (4.6). Burada 10 dosya.
9. **+196 −56** — eklenen ve silinen satır sayısı; kutucuklar oranı gösterir.

4.4 PR sayfası — alt kısım (olaylar)

Aynı sayfanın en altı. Açıklamanın altında PR'ın başına gelen her şey sırasıyla listelenir.



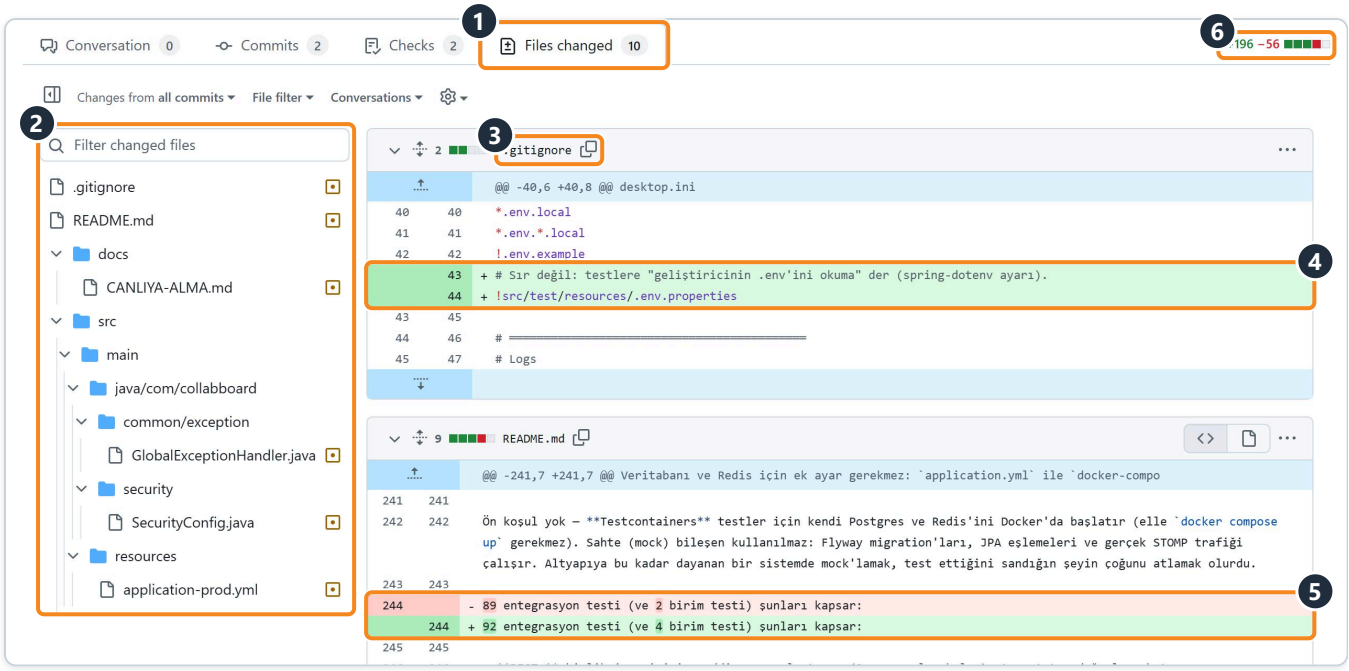
1. **"added 2 commits"** — dala gelen commit'ler, geliş sırasıyla.
2. **✓ 173f7fe.** CI bu commit üzerinde çalıştı, çünkü push anında dalın **en ucundaki** commit oydu. `055c8dc`'nin yanında işaret yok: iki commit tek push'la geldi, CI yalnızca en sondakini dener (en sondaki, öncekileri de içerir).
3. **Merge olayı.** "erdemsidal merged commit `0db46b9` into main" — merge commit'in kimliği ve hedefi.
4. **"2 checks passed".** Merge commit `main`'e girince CI bir kez daha çalıştı (bu kez push: Test + Build) ve ikisi de geçti.
5. **View details** — o iki kontrolün listesi.
6. **Sarı kutu** giriş yapmadan baktığımız için. Sen giriş yaptığında burada yorum kutusu olur; merge'den hemen sonra da `Pull request successfully merged and closed` kutusunu ve `Delete branch` düğmesini görürsün (Bölüm 5 / B).

4.5 Commits sekmesi



1. **Gün başlığı** — commit'ler yazıldıkları güne göre gruplanır.
2. **Commit başlığı** (mesajın ilk satırı). Yanındaki … düğmesi uzun açıklamayı açar.
3. **Yazarlar**: erdemsidal ve claude (commit'teki `Co-Authored-By` satırı sayesinde).
4. **Kısa kimlik** `055c8dc`; yanında kopyala ve <> (o andaki dosyalara göz at). Kimliğe tıklayınca commit sayfası açılır (4.9).
5. **✓ 1 / 2**: Test geçti, Build atlandı. PR'da Build hiç çalışmaz (Bölüm 3), atlanması hata değildir.

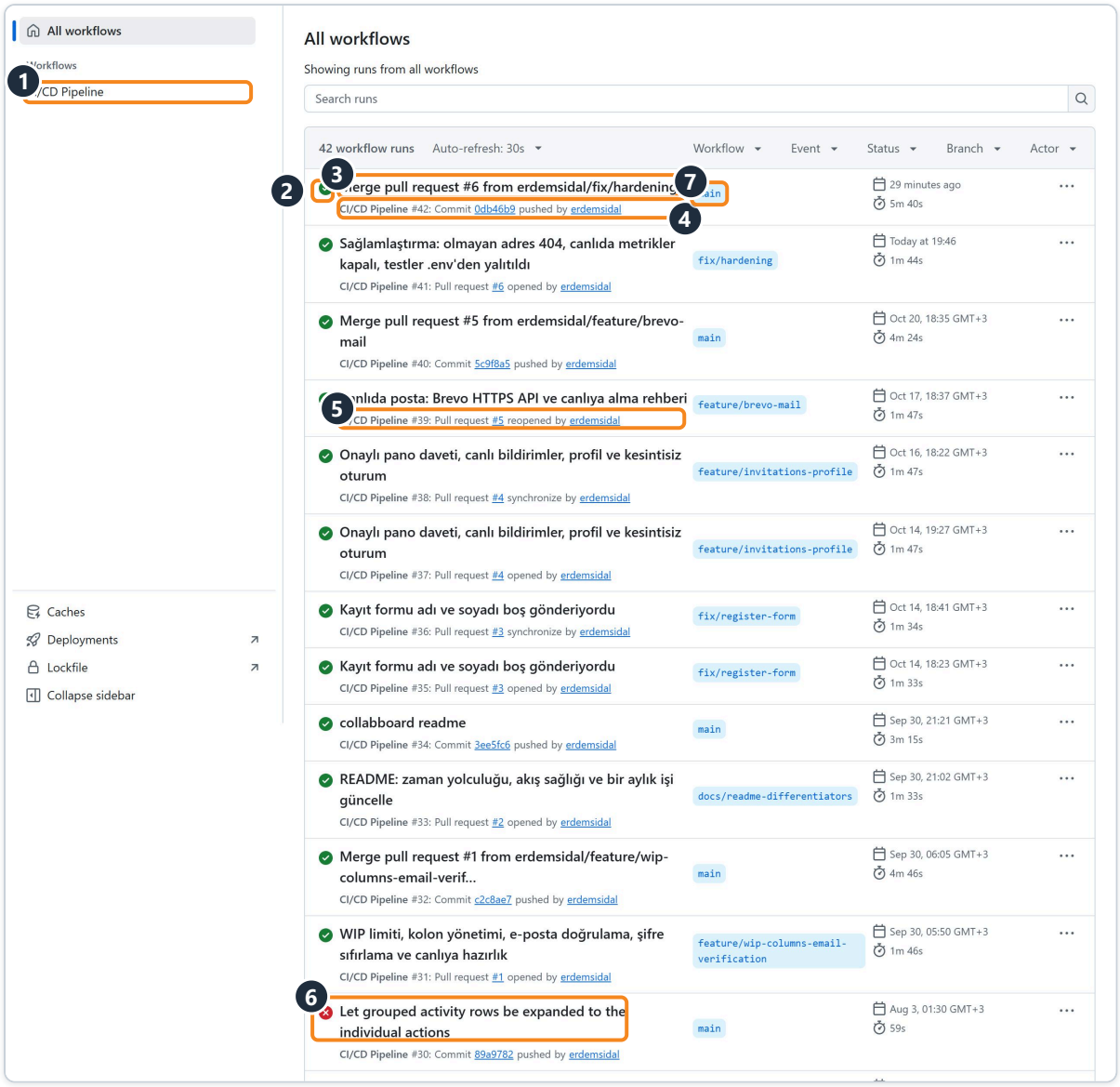
4.6 Files changed sekmesi



1. **Sekme** ve değişen dosya sayısı (10).
2. **Dosya ağacı.** Değişen dosyalar, klasörleriyle. Tıklayınca sağda o dosyaya atlar; sağdaki küçük simge dosyanın durumunu gösterir (değiştirildi / eklendi / silindi).
3. **Dosya başlığı** — `.gitignore`; soldaki "2" bu dosyada değişen satır sayısı.
4. **Yeşil satırlar (+)** eklendi. Soldaki eski satır numarası boş, sağdaki yeni numara dolu.
5. **Kırmızı (−) ve hemen altındaki yeşil (+)**: satır değiştirilmiş = eskisi silinip yenisi yazılmış. Burada README'deki test sayısı 89'dan 92'ye çıkmış.
6. **Toplam**: +196 / −56 satır.

> **Bu sekmeye bakmak zorunda mısın?**
>
> Hayır; ne değiştiğinin özeti PR açıklamasında. Ama "*tam olarak neyi canlıya alıyorum?*" sorusunun kesin cevabı burada. Kod okumasan da dosya ağacına göz atmak (hangi dosyalar? migration var mı?) iyi bir alışkanlık.

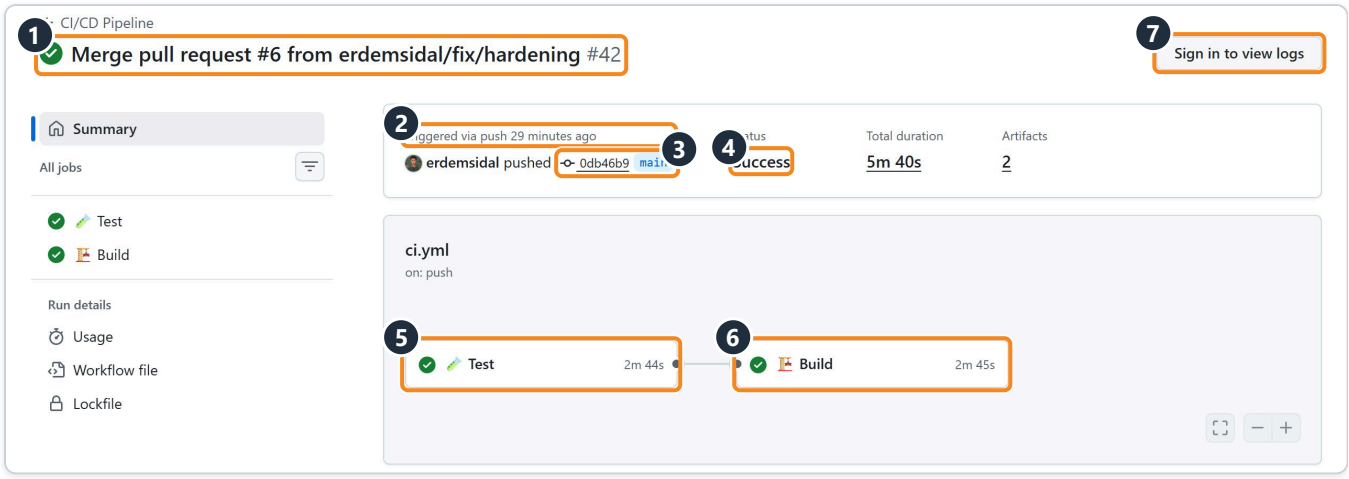
4.7 Actions sekmesi — CI çalışmalarının listesi



1. **Workflow adı**: CI/CD Pipeline (`ci.yml`'deki `name:`). Bizde tek workflow var.
2. **Sonuç**: yeşil ✓ başarılı. Çalışırken sarı/turuncu dönen bir simge görünür.
3. **Başlık**: push'ta commit'in mesajı, PR'da PR'ın başlığı.
4. **"#42: Commit 0db46b9 pushed by erdemsidal"** — çalışma numarası (#42) ve olay: `main`'e push (merge).
5. **"#39: Pull request #5 reopened by erdemsidal"** — PR kapatılıp yeniden açıldığı için çalışan CI. Hikâyesi Bölüm 6'da.
6. **Kırmızı ✗** — 3 Ağustos ve öncesindeki 30 çalışmanın hepsi başarısız (Bölüm 3, "Kırmızı geçen 30 çalışma").
7. **Dal etiketi**: push çalışmalarında `main`, PR çalışmalarında kaynak dal (ör. `fix/hardening`).

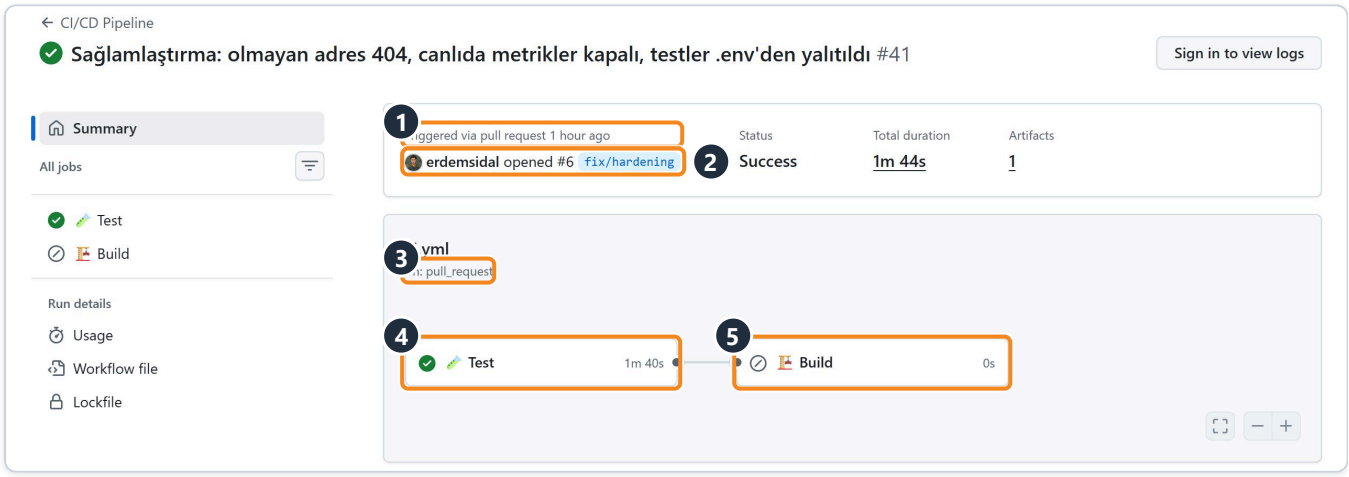
4.8 Bir CI çalışmasının sayfası

Actions listesinde bir satıra tıklayınca açılır. Önce `main`'deki çalışma (#42), sonra aynı işin PR'daki çalışması (#41).



Çalışma #42 — merge'den sonra main'de.

1. **Sonuç, başlık ve numara.**
2. **"Triggered via push"** — neyin başlattığı: `main`'e push (merge).
3. **Hangi commit, hangi dal:** `0db46b9` · `main`.
4. **Status: Success**; yanında toplam süre (5 dk 40 sn) ve üretilen dosya sayısı (Artifacts 2).
5. **Test işi** — 2 dk 44 sn. Giriş yaptığında üstüne tıklayınca adım adım çıktısını (log) görürsün; kırmızıysa hangi testin kırıldığı "Run Tests" adımının sonunda yazar.
6. **Build işi** — Test bitince başladı (aradaki çizgi = `needs: test`). `main`'de olduğu için Docker imajını da üretti.
7. **Sign in to view logs** — loglar yalnızca giriş yapanlara açık.

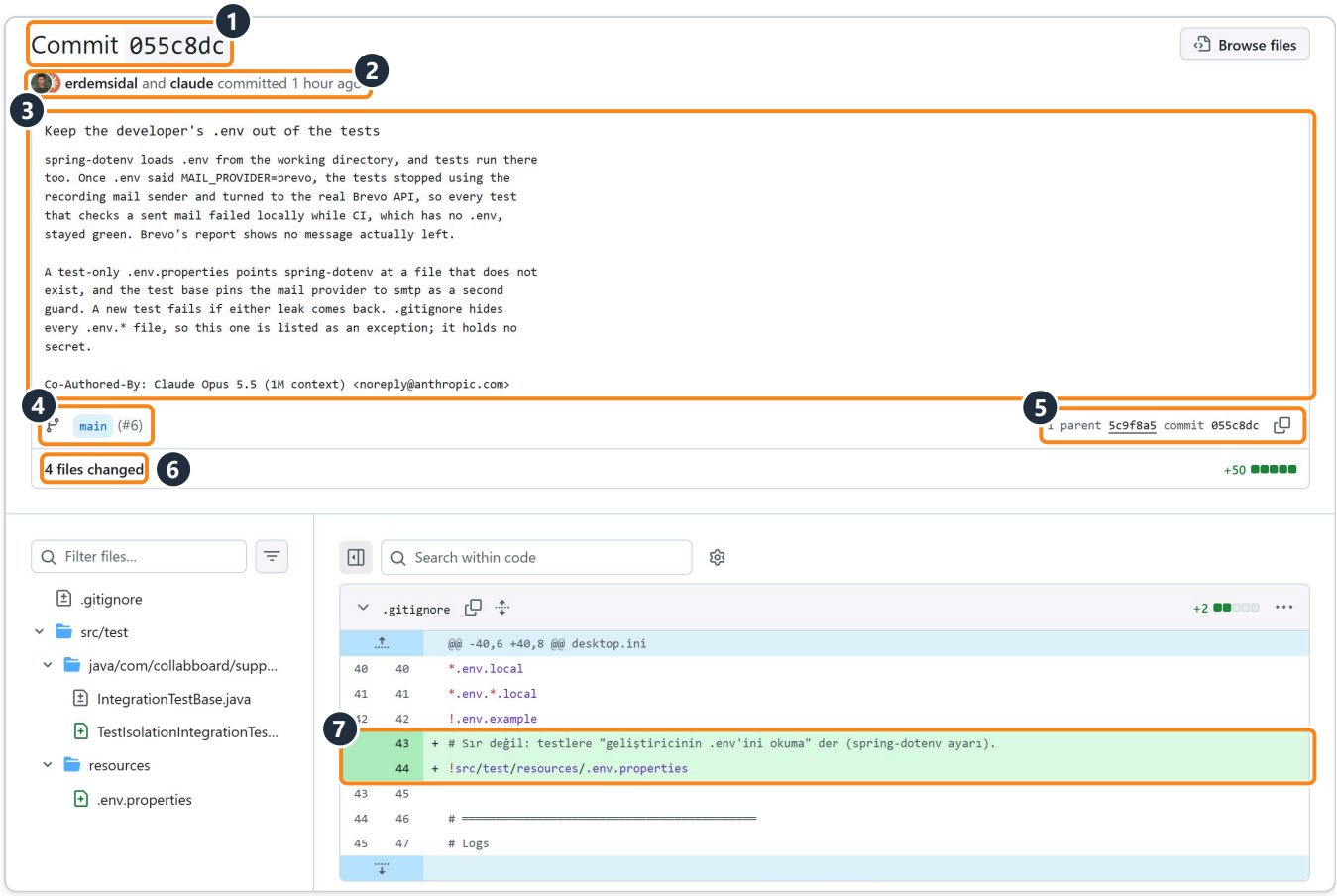


Çalışma #41 — PR #6 açılınca.

1. **"Triggered via pull request"**.
2. **"erdemsidal opened #6"** — olay türü (*opened*) ve kaynak dal.
3. **on: pull_request** — `ci.yml`'in hangi tetikleyicisiyle başladığı.
4. **Test ✓** — 1 dk 40 sn.
5. **Build ⊘ 0s** — atlandı (skipped). PR'da normal; yukarıdaki main çalışmasıyla karşılaştır.

4.9 Commit sayfası

Bir commit kimliğine (ör. `055c8dc`) tıklayınca açılır.



1. **Commit kimliği.**
2. **Yazarlar ve zaman.**
3. **Commit mesajı.** İlk satır başlık; altı "neden"in açıklaması; en sonda `Co-Authored-By`.
4. **main (#6)** — bu commit şu an `main`'de ve PR #6 ile gelmiş.
5. **1 parent 5c9f8a5** — ebeveyn commit: dal açıldığında `main`'in ucu (PR #5'in merge commit'i). Bir merge commit'inde burada "2 parents" yazar.
6. **4 files changed** — bu commit'in değiştirdiği dosya sayısı; solda listesi.
7. **Değişiklik**: `.gitignore`'a iki satır eklenmiş — testlere özel `.env.properties` dosyasının depoya girebilmesi için istisna (Bölüm 6.4).

Bölüm 5 · BİR DEĞİŞİKLİĞİ CANLIYA ALMAK

Bugün kullandığımız akış

Dört parça: **A** hazırlık (Claude), **B** birleştirme (sen), **C** yayın (otomatik), **D** kontrol (sen). Sonda, bu akışın dışına çıkan iki durum: Railway kapalıyken ve yalnızca bir ayar değişirken.

A · Değişiklik bir dalda hazırlanır, PR açılır (Claude yapar)

1. **Güncel `main`'den yeni bir dal açılır** — ör. `git checkout -b fix/hardening`.
   Neden: Değişiklik bitene kadar `main`'e (canlıya) dokunmamak için.
2. **Kod ve testler yazılır, yerelde bütün testler çalıştırılır** — `./mvnw test` (Docker açık).
   Neden: Kırık kodu GitHub'a göndermeden önce yakalamak için.
3. **Commit'lenir** — her commit tek bir konu; İngilizce mesaj, "neden"i anlatır.
   Neden: Altı ay sonra "bu satır neden böyle?" sorusunun cevabı commit mesajıdır.
4. **Push edilir** — `git push -u origin fix/hardening`.
5. **PR açılır** — hedef `main`; açıklamada Neden / Ne değişti / Test / Not.
   Ekranda: Pull requests sekmesinde **Open 1**; PR'da sarı ● ile CI çalışıyor. Claude sana PR bağlantısını verir.
6. **CI kendiliğinden başlar** (pull_request · *opened*), ~2 dakika sürer.
   Ekranda: Birkaç dakika sonra ● yerini yeşil ✓'ye bırakır.

> **PR açıkken yeni bir değişiklik gerekirse**
> Yeni PR açılmaz; Claude aynı dala yeni commit push eder, PR kendiliğinden güncellenir ve CI baştan çalışır (*synchronize*). Sen PR sayfasını yenilediğinde yeni commit'i ve yeni kontrolleri görürsün.

B · Sen birleştirirsin (merge = yayın kararı)

1. **Depoyu aç**: tarayıcıda `github.com/erdemsidal/collab-board` → üstte **Pull requests** sekmesi.
   Ekranda: **Open** listesinde PR başlığı; yanında ✓ (CI geçti) ya da ● (sürüyor).
2. **PR başlığına tıkla.** **Conversation** sekmesi açılır.
   Ekranda: Başlık, yeşil **Open** rozeti, "erdemsidal wants to merge N commits into main from ...", altında açıklama.
3. **Açıklamanın "Not" bölümünü oku.**
   Neden: Birleştirmeden **önce** yapman gereken bir iş varsa orada yazar — ör. PR #5'te "önce Brevo ve değişkenler, sonra veritabanları, en son birleştirme", PR #6'da "önce Railway GitHub uygulaması kurulmalı ve veritabanları açılmalı".
4. **Sayfanın en altına in** (End tuşu). Merge kutusunu bul ve iki satırı kontrol et:
   ✓ All checks have passed ve **✓ This branch has no conflicts with the base branch**.
   Ekranda: Aşağıdaki çizim (a). Kontrollerin ayrıntısını açarsan Test ✓ ve Build "Skipped" görürsün — PR'da normal.
5. **Yeşil Merge pull request düğmesine tıkla.**
   Neden: Düğmenin sağındaki küçük ▾ okuna dokunma: başka birleştirme yöntemleri (*Squash and merge*, *Rebase and merge*) sunar. Bizim düzenimiz varsayılan olan *Create a merge commit*.

6 **Kutu değişir**: merge commit'in başlığı ve açıklaması için iki alan, altında yeşil **Confirm merge** ve **Cancel**. Metinlere dokunmadan **Confirm merge**'e tıkla.

Ekranda: Çizim (b).

Neden: Varsayılan başlık PR numarasını taşır ("Merge pull request #6 from ..."); geçmişte hangi PR'ın ne zaman girdiği buradan okunur. PR #2'de bu alan "collabboard readme" olarak değiştirilmişti; o yüzden `main` geçmişinde #2'nin merge commit'i (`3ee5fc6`) PR numarası taşımıyor.

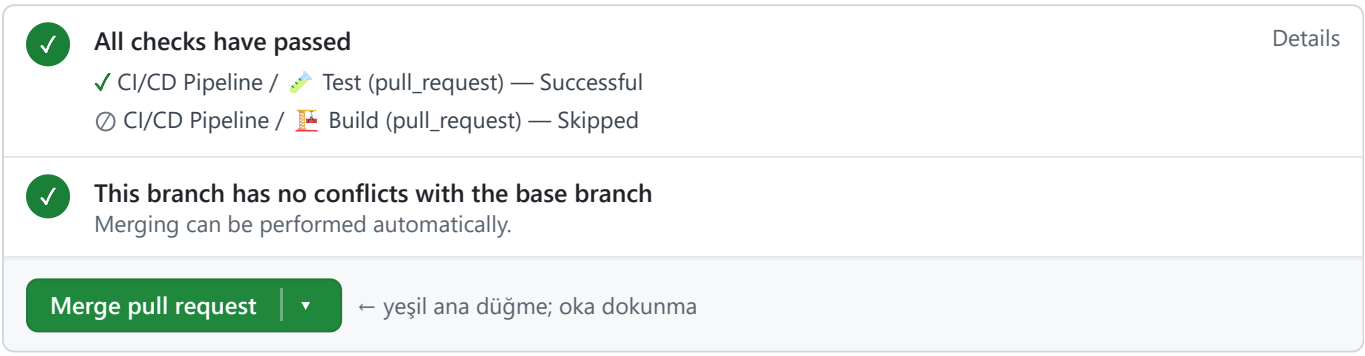
7 **Birkaç saniye bekle.**

Ekranda: Başlıktaki rozet mor **Merged** olur; altta "erdemsidal merged commit ... into main" satırı ve **Pull request successfully merged and closed** kutusu çıkar — çizim (c).

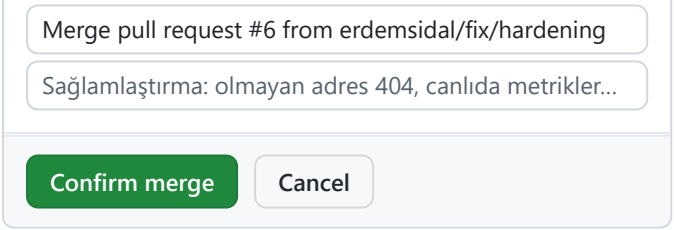
8 **İsteğe bağlı: Delete branch.**

Neden: Dalı GitHub'dan siler; commit'ler `main`'de kalır. Gerekirse aynı yerden **Restore branch**. Silmesen de hiçbir şey bozulmaz.

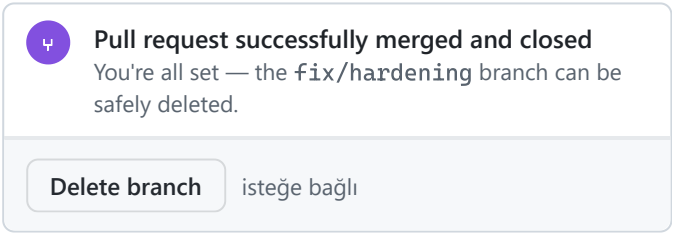
TEMSİLİ ÇİZİM (a) — birleştirmeye hazır PR'ın merge kutusu



TEMSİLİ ÇİZİM (b) — onay



TEMSİLİ ÇİZİM (c) — birleştikten sonra



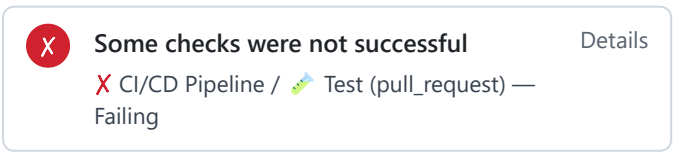
GitHub görünümünü zaman zaman yeniler; kutuların yerleşimi biraz farklı olabilir, başlık ve düğme yazıları aynı kalır.

Ne zaman birleştirmezsin?

GitHub seni durdurmaz

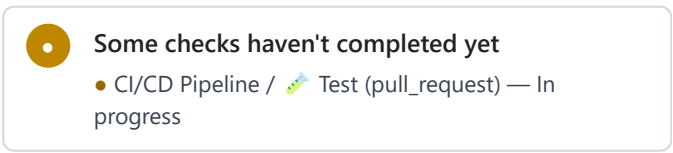
`main` dalında koruma kuralı (branch protection) **yok**. Yani kontroller kırmızıyken de **Merge pull request** düğmesi basılabilir durumdadır. Karar tamamen sende: aşağıdaki durumlardan birini görürsen **basma**.

(d) KIRMIZI ✗ — test kırılmış



Yap: birleştirme. Claude'a "PR #N'de CI kırmızı" yaz. Düzeltme aynı dala push edilince CI kendiliğinden yeniden çalışır.

(e) SARI ● — CI sürüyor



Yap: 2–3 dakika bekle, sayfayı yenile (F5).

(f) ÇAKIŞMA

> ! **This branch has conflicts that must be resolved**
> Conflicting files: README.md (örnek)
>
> Merge pull request kullanılamaz

Yap: Claude'a "PR #N'de conflict var" yaz; dalı güncel `main`'le birleştirip çözer.

(g) DİĞER "DUR" İŞARETLERİ

- Rozet Draft: PR taslak, henüz hazır değil.
- "Not"ta önce yapılacak bir iş yazıyor ve henüz yapmadın.
- Railway'de servisler durdurulmuş (Remove) — önce aşağıdaki "İstisna 1".
- PR'ın hedefi `main` değil (üstte "into *başka-dal*").

C · Otomatik kısım: GitHub → Railway

Confirm merge'e bastığın an iki şey aynı anda olur:

1. **GitHub Actions** `main`'deki yeni merge commit için CI'ı başlatır (Test, ardından Build; toplam ~5–6 dk).
2. **GitHub, Railway'e haber verir** ("zil", teknik adı *webhook*). Bu haber, GitHub hesabına kurulu **Railway App** üzerinden gider. Railway yeni bir yayın (deployment) açar.

Railway'de `collab-board` kutusu → Deployments sekmesinde en üstte yeni bir satır belirir ve şu durumlardan geçer:

Durum	Ne oluyor?
Waiting for CI	Railway, GitHub'daki kontrollerin (Test + Build) bitmesini bekliyor. **"Wait for CI"** ayarı açık olduğu için: kontroller kırmızı biterse bu sürüm yayına **alınmaz**.
Initializing	CI ✓ geldi; yayın hazırlanıyor.
Building	Kod GitHub'dan alınıyor, `Dockerfile` ile imaj derleniyor. Derleme loglarında Maven çıktısını görürsün.
Deploying	Yeni konteyner açılıyor: Variables verilir, Spring Boot başlar, Flyway yeni migration'ları uygular.
ACTIVE · Online	Trafik yeni sürüme döndü. Satırda ACTIVE, servis kutusunda yeşil **Online**. Önceki sürüm kaldırılır.

TEMSİLİ ÇİZİM — Railway · collab-board · Deployments

> ACTIVE **Merge pull request #6 from erdemsidal/fix/hardening** View logs ⋮
> main · 0db46b9
>
> REMOVED Merge pull request #5 from erdemsidal/feature/brevo-mail ⋮
> main · 5c9f8a5

Railway paneli giriş gerektirdiği için ekran görüntüsü yok; durum adları ve düğmeler paneldeki gibidir.

> **Eski sürüm ne zaman kalkar?**
> Yeni sürüm hazır olana kadar **eski sürüm çalışmaya devam eder**; ziyaretçi merge anında bir şey fark etmez. Bir istisna: Railway'de Settings → Deploy → *Healthcheck Path* = `/actuator/health` girilmemişse Railway, yeni konteyner açılır açılmaz trafiği ona verir; uygulama hazır olana kadar ~20 saniye "Application failed to respond" görülebilir. Bu ayar bir kez girilir (CANLIYA-ALMA.md, Adım 5); girilip girilmediğini aynı yerden görebilirsin.

> **"Zil" nerede kurulu? (23 Ekim'de kuruldu)**
> **GitHub tarafı:** sağ üstte profil fotoğrafın → **Settings** → sol menüde **Applications** → **Installed GitHub Apps** → **Railway** → **Configure** → *Repository access*'te `collab-board` seçili olmalı.
> **Railway tarafı:** `collab-board` servisi → **Settings** → **Source**: bağlı depo `erdemsidal/collab-board`, dal `main`; **Auto deploys when pushed to GitHub** açık; **Wait for CI** açık.
> **Kanıt:** Railway her yayında GitHub'a bir "Deployment" kaydı yazar. GitHub'da Actions sekmesinin sol altındaki **Deployments** bağlantısından görülür; 23 Ekim'den önce hiç kayıt yoktu.

D · Kontrol (sen yaparsın, ~2 dk)

1. **Railway'de yayının bittiğini gör**: railway.com → `collab-board` projesi → `collab-board` kutusu → **Deployments**.
   Ekranda: En üst satırda `ACTIVE` ve az önceki merge'ün başlığı ("Merge pull request #N ...").
2. **Logları aç**: o satırda **View logs**. Arama kutusuna sırayla yaz:
   - `Started` — uygulamanın açıldığı an.
   - `migration` — hangi veritabanı değişikliklerinin uygulandığı (varsa).
   - `Brevo` — "Brevo bağlantısı doğrulandı", "Brevo gönderen adresi onaylı".

   Neden: Her log satırında `level_value` alanı var: **20000** = INFO (bilgi), 30000 = WARN (uyarı), **40000** = ERROR (hata). 40000 görürsen satırı oku; kutu içinde büyük harfli bir uyarı varsa CANLIYA-ALMA.md Bölüm 6'da aynı başlığı ara.
3. **Siteyi aç ve `Ctrl` + `F5` ile yenile.**
   Neden: `Ctrl`+`F5` tarayıcının önbelleğini atlayıp sayfayı sunucudan yeniden ister; eski sürümü görme ihtimalini sıfırlar.
4. **Değişikliği gözünle dene.** İstersen hızlı sağlık testleri: `/actuator/health` → `{"status":"UP"}`; `/actuator/metrics` → 404 (PR #6'dan beri kapalı).
5. **GitHub'da son bir bakış**: depo ana sayfasında son commit'in yanında ✓; Actions'ta en üst `main` çalışması yeşil.

İstisnalar

İstisna 1 — Railway durdurulmuşsa (Remove ile kapatıldıysa)

Kredi harcamasın diye servisleri **Remove** ile kapattıysan, merge'den **önce** veritabanlarını aç:

1. Railway'de `Postgres` kutusu → **Deployments** → en üstteki satır (`REMOVED` yazar) → sağdaki ⋮ → **Redeploy**.
2. Aynısını `Redis` için yap. İkisinde de yeşil **Online** görünene kadar bekle.
3. Şimdi PR'ı birleştir (B). Railway uygulamayı kendisi açar. Birleştirilecek PR yoksa: `collab-board` → **Deployments** → ⋮ → **Redeploy**.
   Neden: Uygulama açılırken Postgres'e ve Redis'e bağlanır; onlar kapalıysa çöker ve Railway tekrar tekrar açmaya çalışıp kredi harcar.

Kapatırken sıra tersidir: önce `collab-board`, sonra `Postgres` ve `Redis` — her birinde **Deployments** → `ACTIVE` satırı → ⋮ → **Remove**. *Remove silmek değildir*: yalnızca çalışan kopya kalkar; kod, ayarlar, veriler ve adres yerinde durur.

İstisna 2 — Yalnızca bir ayar (Variable) değişiyorsa: PR gerekmez

1. `collab-board` kutusu → **Variables**.
2. Değeri değiştir ya da **+ New Variable** ile ekle (sol kutuya ad, sağa değer → **Add**). Aynı ad varsa "Variable overwrite detected" → **Overwrite**.
3. Sayfanın üstünde **Apply N changes** çubuğu çıkar → **Deploy**.
   Neden: Ayarlar git'te değil Railway'de durur; kod değişmediği için GitHub'a, PR'a, CI'a gerek yok. Railway aynı kodu yeni ayarlarla yeniden başlatır. Gizli değerler (ör. `BREVO_API_KEY`, `JWT_SECRET`) yalnızca burada yaşar.

Otomatik yayın başlamazsa / geri almak gerekirse

- **Başlamazsa:** Railway sayfasında `Ctrl` + `K` → `deploy` yaz → **Deploy Latest Commit** → servis olarak `collab-board`. Sonra "zil"i kontrol et (yukarıdaki kutu; Bölüm 7).
- **Hızlı geri alma:** Railway → **Deployments** → çalışan son iyi sürüm → ⋮ → **Redeploy**. Kod eski hâline döner; **veritabanı dönmez**.
- **Kalıcı geri alma:** GitHub'da sorunlu PR'ın sayfasında **Revert** düğmesi → yeni bir PR açılır → onu birleştir → Railway otomatik yayınlar.

Bölüm 6 · GERÇEK GEÇMİŞİMİZ

PR #1'den PR #6'ya

Bu bölümdeki her tarih ve saat GitHub kayıtlarından (git geçmişi, PR olayları, Actions çalışmaları, yayın kayıtları) alındı.

27 Tem – 3 Ağu · **PR'sız dönem.** 30 commit doğrudan `main`'e yazıldı: pano, gerçek zamanlı senkron, çakışma çözümü, Redis köprüsü, kimlik doğrulama, roller, çalışma alanları, yeni arayüz, zaman yolculuğu. CI çalışmaları #1–#30'un hepsi kırmızı (Bölüm 3).

30 Eyl 05:50 → 06:05 · **PR #1** `feature/wip-columns-email-verification` — 11 commit: WIP limiti, kolon yönetimi, e-posta doğrulama, şifre sıfırlama, canlıya hazırlık, **CI düzeltmesi**. CI #31: ilk yeşil. Merge commit `c2c8ae7`.

30 Eyl 21:02 → 21:21 · **PR #2** `docs/readme-differentiators` — yalnızca README. Merge commit mesajı "collabboard readme" olarak değiştirildi (`3ee5fc6`).

1 Eki · **Railway** deneme planı başladı ($5 / 30 gün) — CANLIYA-ALMA.md 6.7.

14 Eki 18:23 · **PR #3** `fix/register-form` — kayıt formu adı ve soyadı boş gönderiyordu.

14 Eki 19:27 · **PR #4** `feature/invitations-profile` — #3'ün dalının **üstüne** kuruldu: onaylı davet, canlı bildirim, profil; 16 Eki'de kesintisiz oturum eklendi.

17 Eki 18:35 → 18:37 · **PR #5** `feature/brevo-mail` — #4'ün üstüne. Hedefi önce `feature/invitations-profile`; 18:36'da `main`'e çevrildi; CI çalışmadı; 18:37'de kapatılıp yeniden açıldı → CI #39 ✓.

20 Eki 18:35 · **#5 birleştirildi** (`5c9f8a5`); #3 ve #4 aynı anda kendiliğinden "merged" oldu. Railway **yayın başlatmadı** → elle "Deploy Latest Commit". Canlıda posta Brevo ile çalıştı; sonra servisler Remove ile durduruldu. Aynı gün: Railway CLI'nin bu bilgisayarda engellendiği görüldü.

23 Eki · **Zil kuruldu**: Railway App GitHub'a kuruldu, "Auto deploys" ve "Wait for CI" açıldı.

23 Eki 19:46 → 20:46 · **PR #6** `fix/hardening` — 20:39 merge, 20:45 CI ✓, 20:46 canlı. **İlk otomatik yayın.**

6.1 Bütün PR'lar bir bakışta

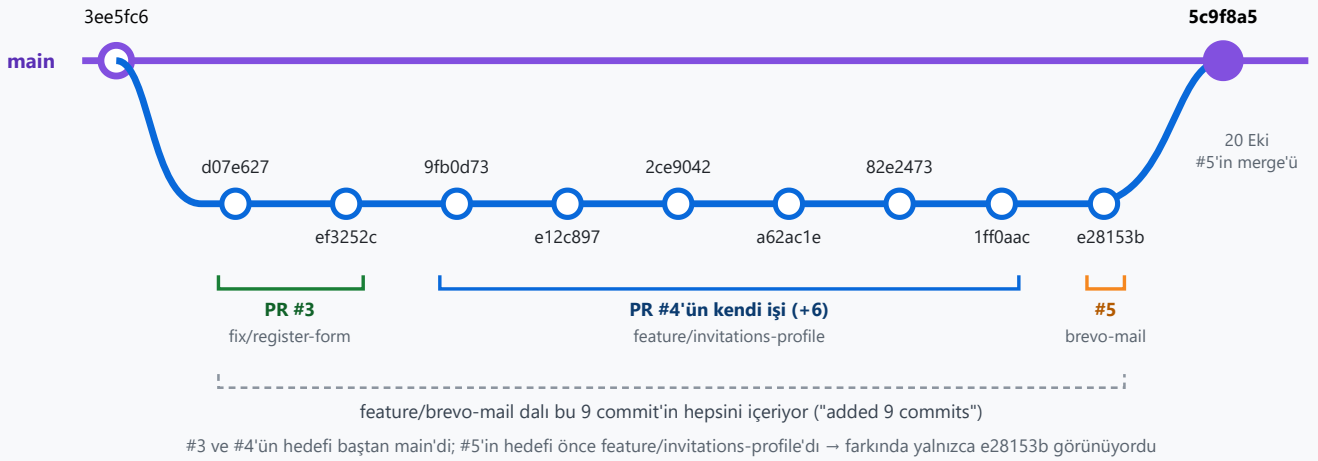
#	Başlık	Açıldı → birleşti	Ne değişti?
1	WIP limiti, kolon yönetimi, e-posta doğrulama, şifre sıfırlama ve canlıya hazırlık	30 Eyl 05:50 → 06:05	Dolu kolona kart girmez; kolon ekle/adlandır/sil; yaşlanan iş; arama/süzme; hesap e-postayla doğrulanır; şifre sıfırlama; güvenlik (origin listesi, prod'da açık JWT anahtarıyla açılmama, hız sınırı); CI düzeldi; `PORT`/Redis parolası; `docker compose` ile tüm yığın. 11 commit.
2	README: zaman yolculuğu, akış sağlığı ve bir aylık işi güncelle	30 Eyl 21:02 → 21:21	README'ye ürünü farklı kılan iki özellik, demo, API ve ADR tablosu, CI rozeti. Kod yok.
3	Kayıt formu adı ve soyadı boş gönderiyordu	14 Eki 18:23 → 20 Eki 18:35	Form değerleri durumda tutuluyor; hata sonrası form silinmiyor; nötr örnek yazılar; "Benimle paylaşılanlar" bölümü.

4	Onaylı pano daveti, canlı bildirimler, profil ve kesintisiz oturum	14 Eki 19:27 → 20 Eki 18:35	Davet → kabul ile üyelik; kişiye özel canlı bildirim; profil (ad, şifre, fotoğraf); 15 dakikada oturumdan atma hatası (403 → 401); 30 günlük kayan oturum; saat dilimi düzeltmesi.
5	Canlıda posta: Brevo HTTPS API ve canlıya alma rehberi	17 Eki 18:35 → 20 Eki 18:35	Railway SMTP portlarını kapattığı için posta Brevo'nun HTTPS API'sinden; `MAIL_PROVIDER`; açılışta anahtar/gönderen kontrolü; `docs/CANLIYA-ALMA.md`, `docs/MAIL-KURULUMU.md`.
6	Sağlamlaştırma: olmayan adres 404, canlıda metrikler kapalı, testler .env'den yalıtıldı	23 Eki 19:46 → 20:39	`/favicon.ico` ve kapalı Swagger 500 yerine 404; canlıda yalnızca `/actuator/health` açık; testler geliştiricinin `.env`'ini okumuyor; rehber Railway paneline göre.

Değişikliklerin ayrıntılı listesi ve "nasıl denenir" notları: `docs/DEGISIKLIK-GUNLUGU.md`.

6.2 Üst üste kurulan PR'lar: #3 → #4 → #5

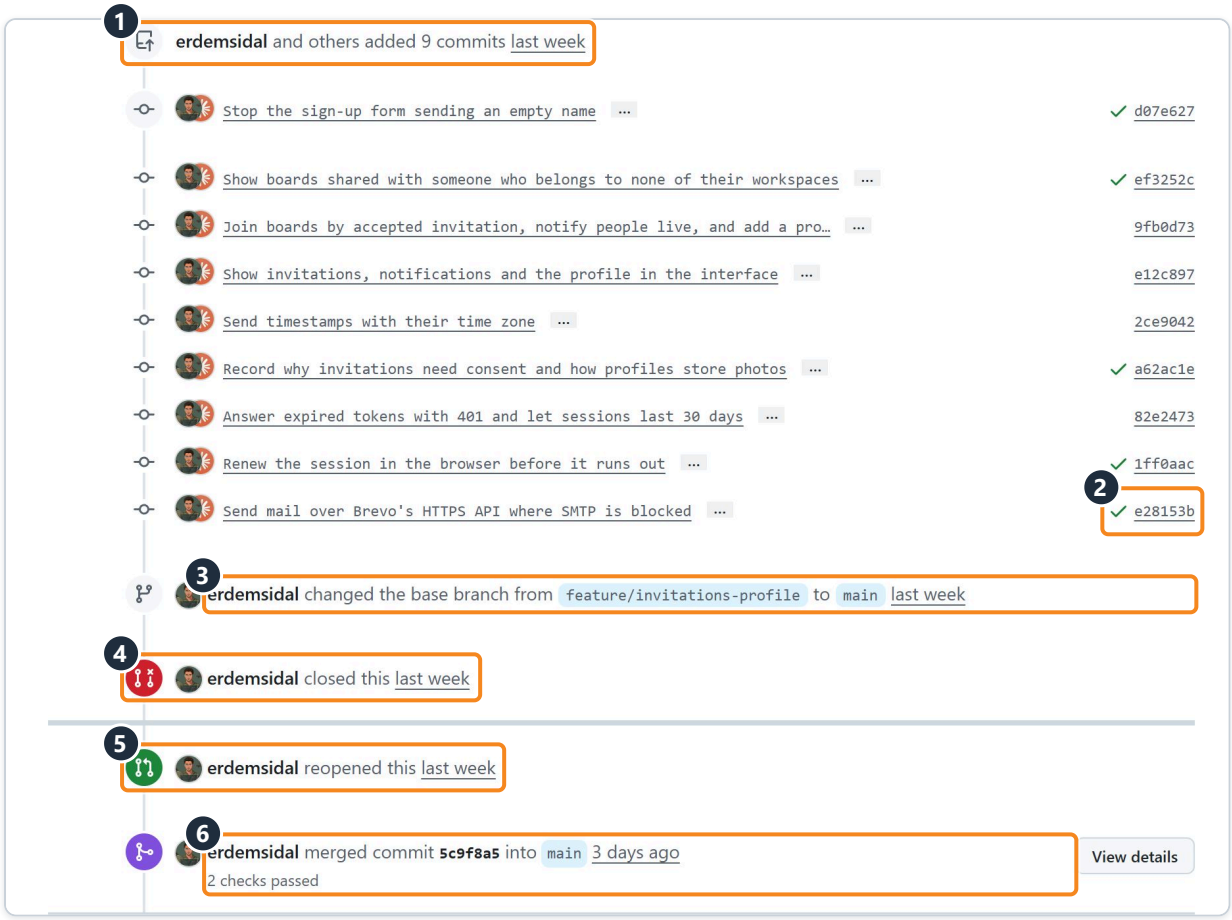
#3 henüz birleştirilmemişken iş devam etti. #4'ün dalı #3'ün dalından, #5'in dalı da #4'ün dalından açıldı; yani her dal bir öncekinin bütün commit'lerini içeriyordu. Buna **"stacked PR"** (üst üste PR) denir.



Neden #5'in hedefi `main`'e çevrildi? İki sebeple: (1) CI yalnızca hedefi `main` olan PR'larda çalışıyor (`pull_request: branches: [main]`); hedefi `feature/invitations-profile` iken #5'te hiç test koşmamıştı. (2) Hedef başka bir dal olsaydı, birleştirmek kodu o dala götürürdü, `main`'e (canlıya) değil.

Birleştirme günü ne oldu? 20 Ekim 18:35:53'te #5 birleştirildi; tek merge commit (`5c9f8a5`) dokuz commit'in hepsini `main`'e taşıdı. GitHub, #3'ün ve #4'ün commit'lerinin artık `main`'de olduğunu görünce iki saniye sonra (18:35:55) onları da kendiliğinden "merged" işaretledi. Bu yüzden `main` geçmişinde "Merge pull request #3" ya da "#4" diye ayrı commit yok.

6.3 Hedef değişince CI neden çalışmadı?



PR #5'in Conversation sekmesinin alt kısmı (gerçek ekran görüntüsü).

1. **"added 9 commits"** — dal, #3 ve #4'ün commit'lerini de taşıyor.
2. **✓ e28153b** — kapatıp açtıktan sonra çalışan CI'ın işareti. Diğer ✓'ler (`d07e627`, `ef3252c`, `a62ac1e`, `1ff0aac`) #3'ün ve #4'ün kendi CI çalışmalarından.
3. **"changed the base branch from feature/invitations-profile to main"** — 17 Eki 18:36. GitHub bunu *edited* olayı sayar.
4. **"closed this"** — 18:37:32.
5. **"reopened this"** — 18:37:34. *reopened* olayı CI'ı başlattı: Actions'ta çalışma #39 "Pull request #5 reopened by erdemsidal" (Bölüm 4.7, işaret 5).
6. **"merged commit 5c9f8a5 into main"** — 20 Eki, "2 checks passed".

`ci.yml`'deki `pull_request` tetikleyicisi olay türü belirtmiyor; GitHub'ın varsayılanı yalnızca **opened**, **synchronize** ve **reopened**. PR açıldığında hedef `main` değildi, filtre tutmadı; hedef `main`'e çevrildiğinde ise olay **edited** oldu ve listede yok. İki çıkış yolu vardı: dala yeni bir commit push etmek (synchronize) ya da PR'ı kapatıp yeniden açmak (reopened). Commit eklememek için ikincisi seçildi.

> **Sen de yapabilirsin**
> PR sayfasının en altında yorum kutusunun yanında **Close pull request** düğmesi var; basınca aynı yerde **Reopen pull request** çıkar. Kapatıp açmak hiçbir şeyi silmez; yalnızca olay üretir.

6.4 Birleştirdik ama yayın başlamadı (20 → 23 Ekim)

- **Belirti:** 20 Ekim'de #5 birleşti, `main`'deki CI (#40) yeşil bitti, ama Railway'de hiçbir şey olmadı. Geçici çözüm olarak Railway'de `Ctrl`+`K` → **Deploy Latest Commit** ile elle yayınlandı.

- **Sebep:** Railway kodu *okuyabiliyordu* (depo herkese açık), ama yeni kodun geldiğini *duymuyordu*. GitHub'ın bir uygulamaya "main'e kod girdi" diye haber verebilmesi için o uygulamanın (Railway App) GitHub hesabına kurulu ve depoya erişiminin açık olması gerekir. Kurulu değildi.
- **Teşhis nasıl konuldu?** Railway App kuruluyken her yayında GitHub'a bir "Deployment" kaydı yazar. GitHub'da bu depo için **sıfır** Deployment kaydı vardı → Railway ile GitHub arasında hiç bağ kurulmamıştı.
- **Çözüm (23 Ekim):** GitHub hesabına **Railway App** kuruldu, `collab-board` deposuna erişim verildi; Railway'de **Auto deploys when pushed to GitHub** ve **Wait for CI** açıldı.
- **Doğrulama:** PR #6 birleşince (20:39) Railway ilk kez kendiliğinden yayın açtı ve GitHub'a kayıt yazdı (20:39:42, "Waiting for CI"); CI 20:45'te bitti; yeni sürüm 20:46'da canlıdaydı (kayıt 20:46:51 "success"). Canlıda `/actuator/metrics` ve `/favicon.ico` artık 404.

6.5 Geliştiricinin `.env`'i testlere sızıyordu (PR #6)

- **Ne oldu?** PR #1'de eklenen `spring-dotenv`, `.env` dosyasını çalışma klasöründen okur — testler de aynı klasörde çalışır. Brevo'yu yerelde denemek için `.env`'e `MAIL_PROVIDER=brevo` yazılınca testler sahte posta kutusu yerine gerçek Brevo'ya yöneldi ve **yerelde 11 test kırıldı**. CI'da `.env` olmadığı için **CI yeşil kaldı**. Brevo raporuna göre gerçekte posta gitmedi.
- **Düzeltme:** Testlere özel `src/test/resources/.env.properties` dosyası `spring-dotenv`'e var olmayan bir dosya adı veriyor (`filename=.env.testlerde-okunmaz`, `ignoreIfMissing=true`); test tabanı posta yolunu ayrıca `smtp`'ye sabitliyor; yeni `TestIsolationIntegrationTest` sızıntı geri gelirse kırılıyor. `.gitignore` bütün `.env.*` dosyalarını gizlediği için bu dosyaya istisna eklendi (içinde sır yok) — Bölüm 4.9'daki yeşil satırlar.
- **Ders:** "Yerelde kırmızı, CI'da yeşil" (ya da tersi) neredeyse her zaman **ortam farkıdır**: bir makinede olan bir dosya, ayar ya da program diğerinde yoktur.

6.6 Railway CLI bu bilgisayarda çalışmıyor

Windows'un **Akıllı Uygulama Denetimi** (Smart App Control) `railway.exe`'yi engelliyor: komut hiçbir şey yazmadan kapanıyor. Bu yüzden Railway'deki her iş **railway.com web panelinden** yapılıyor ve `docs/CANLIYA-ALMA.md` PR #6'da panel adımlarına göre yeniden yazıldı (CLI karşılıkları belgenin içinde, katlanmış bölümde duruyor).

Bölüm 7 · SIK KARŞILAŞILAN DURUMLAR

Belirti → sebep → ne yapmalı

Önce gördüğün şeyi sol sütunda bul. Canlıdaki uygulamaya özgü hatalar (posta, açılış, oturum) için ayrıca `docs/CANLIYA-ALMA.md` Bölüm 6.

Belirti	Sebep	Ne yapmalı
PR'da sarı ● / "Some checks haven't completed yet"	CI hâlâ çalışıyor (PR'da ~2 dk).	Bekle, sayfayı yenile. Birleştirme.
PR'da kırmızı ✗ / "Some checks were not successful"	Bir test kırıldı ya da derleme başarısız.	**Birleştirme** (GitHub seni durdurmaz). Claude'a haber ver; düzeltme push edilince CI kendiliğinden yeniden çalışır. Ayrıntı: **Details** → giriş yaptıysan log.
PR'da Build "Skipped" / commit yanında "1 / 2"	Build yalnızca push'ta çalışır.	Hiçbir şey; normal (Bölüm 3).
Çalışma sayfasında sarı üçgenli "Annotations" (Node.js 20, setup-java v4, Ubuntu 26)	GitHub'ın bakım uyarıları.	Şimdilik yok say; ileride sürüm güncelleyen küçük bir PR.
"This branch has conflicts that must be resolved"	Aynı satırlar hem dalda hem `main`'de değişmiş.	Birleştirme; Claude'a söyle. Dal güncellenip çakışma çözülür.
PR'ın hedefini değiştirdim, CI çalışmadı	Hedef değiştirmek *edited* olayıdır; CI yalnızca opened / synchronize / reopened'da çalışır.	**Close pull request** → **Reopen pull request** (ya da yeni bir commit push). Bölüm 6.3.
PR'da hiç kontrol görünmüyor	PR'ın hedefi `main` değil.	Başlığın altındaki "into …" kısmına bak. Hedef `main` olmalı.
Birleştirdim, CI yeşil, ama Railway hiç yayın başlatmadı	"Zil" yok: Railway App kurulu değil ya da depoya erişimi yok. İzi: GitHub'da yeni Deployment kaydı oluşmadı.	GitHub → profil → Settings → Applications → Installed GitHub Apps → Railway → Configure → `collab-board` seçili mi? Railway → collab-board → Settings → Source. O ana kadar: `Ctrl`+`K` → Deploy Latest Commit.
Railway uzun süre "Waiting for CI"	`main`'deki CI (Test + Build, ~6 dk) bitmedi.	GitHub → Actions'ta en üstteki `main` çalışmasını izle. Kırmızı biterse Railway bu sürümü yayınlamaz: önce kodu düzelt.
Yayından sonra servis "Crashed"	Çoğunlukla Postgres/Redis kapalı ya da bir değişken eksik/yanlış.	**View logs**; CANLIYA-ALMA.md 6.2'deki tablo. Veritabanları kapalıysa önce onları **Redeploy**.
Yayından hemen sonra ~20 sn "Application failed to respond"	Healthcheck Path girilmemiş: trafik, uygulama hazır olmadan yeni sürüme döndü.	Bir kez: Settings → Deploy → Healthcheck Path = `/actuator/health`.
Site hâlâ eski görünüyor	Yayın henüz bitmedi ya da tarayıcı eski sayfayı gösteriyor.	Railway'de en üst satır **ACTIVE** mı ve doğru merge'ün başlığını mı taşıyor? Sonra `Ctrl`+`F5`.
Değişken değiştirdim, etkisi yok	Değişiklik biriktirildi ama yayınlanmadı.	Variables sayfasının üstündeki **Apply N changes** → **Deploy**.
Yerelde testler kırmızı, CI yeşil	Ortam farkı (yerel `.env`, Docker kapalı, eski bağımlılık…).	Docker Desktop açık mı? `.env`'de yeni bir ayar var mı? PR #6 örneği: Bölüm 6.5.

Belirti	Sebep	Ne yapmalı
`railway` komutu hiçbir şey yazmadan kapanıyor	Windows Akıllı Uygulama Denetimi engelliyor.	Web panelini kullan (railway.com).
Yanlışlıkla birleştirdim / yayın bir şeyi bozdu	—	Hızlı: Railway → Deployments → önceki iyi sürüm → ⋮ → **Redeploy** (veritabanı geri dönmez). Kalıcı: PR sayfasında **Revert** → yeni PR → birleştir.
"Delete branch"e bastım, pişman oldum	—	Aynı PR sayfasında **Restore branch**. Commit'ler zaten `main`'de.
Yerelde açtığım hesap canlıda yok	Yerel veritabanı ile canlı veritabanı ayrı.	Canlıda yeniden kaydol. İkisi hiçbir zaman eşitlenmez.

Bölüm 8 · SÖZLÜK

Terimler

ACTIVE
Railway'de şu an trafiği karşılayan yayın.

Actions (GitHub Actions)
GitHub'ın iş çalıştıran servisi; CI'ımız burada koşar.

Annotations
Çalışma sayfasındaki uyarı/bilgi notları. Sarı üçgen = uyarı, hata değil.

Artifact
Bir CI çalışmasının ürettiği, indirilebilir dosya (ör. `test-reports`).

Base (hedef dal)
PR'ın birleşeceği dal; bizde `main`.

Branch (dal)
Bir commit'i gösteren, adı olan etiket.

Branch protection
"Kırmızıyken birleştirilemesin" gibi kurallar. Bizde `main` için kapalı.

Build
CI'daki ikinci iş; JAR + Docker imajı. Yalnızca push'ta çalışır.

Check (kontrol)
Bir CI işinin sonucu: ✓ ✗ ● ⊘.

Checkout
Kodu (bir dalı/commit'i) çalışma klasörüne çıkarmak; CI'ın ilk adımı.

CI
Her değişiklikte temiz makinede otomatik derleme + test.

Commit
Dosyaların bir anlık fotoğrafı + mesaj + ebeveyn bağlantısı.

Conflict (çakışma)
Aynı satırların iki dalda farklı değişmesi; elle çözülür.

Deployment (yayın)
Bir sürümün sunucuda açılması. Railway her birini listeler; GitHub'a da kayıt yazar.

Diff (fark)
İki sürüm arasındaki satır farkı; yeşil +, kırmızı −.

Draft
Taslak PR; birleştirmeye hazır değil.

Fetch
GitHub'daki yenilikleri indirmek, ama yerel dala katmamak.

Flyway / migration
Veritabanı şemasını sırayla değiştiren dosyalar (`V13__…`); açılışta uygulanır, geri alınmaz.

ghcr.io
GitHub'ın Docker imaj deposu; Build iş imajı oraya gönderir.

GitHub App
GitHub hesabına kurulan, belirli depolara erişen uygulama (ör. Railway App). "Zil" bunun üzerinden çalar.

Hash (commit kimliği)
Commit'in 40 haneli özeti; kısaca ilk 7 hane.

Head (kaynak dal)
PR'daki değişikliklerin bulunduğu dal.

level_value
Railway logunda önem derecesi: 20000 INFO, 30000 WARN, 40000 ERROR.

main
Ana dal. Bizde = canlı.

Merge
Bir dalın commit'lerini başka bir dala katmak.

Merge commit
İki ebeveynli, birleştirmeyi kaydeden commit.

origin
Git'in GitHub'daki uzak kopyaya verdiği varsayılan ad.

Public
Depo herkese açık: okuma herkese, yazma yalnızca sahibine.

Pull
GitHub'daki yeni commit'leri yerel dala almak (fetch + merge).

Pull request (PR)
Kaynak daldan hedef dala birleştirme isteği ve bunun GitHub sayfası.

Push
Yerel commit'leri GitHub'a göndermek.

Rebase / Squash
Başka birleştirme yöntemleri; bizde kullanılmıyor.

Redeploy
Railway'de bir yayını (aynı kodla) yeniden açmak.

Remove
Railway'de çalışan kopyayı kaldırmak; veri ve ayar silinmez.

Repository (depo)
Dosyalar + bütün commit geçmişi.

Revert
Bir değişikliği geri alan *yeni* bir commit/PR; geçmiş silinmez.

Runner
CI'ı çalıştıran GitHub makinesi (bizde `ubuntu-latest`).

Skipped (atlandı)
Koşulu tutmadığı için çalışmayan iş; hata değil.

Stacked PR
Bir PR'ın dalının başka bir PR'ın dalı üstüne kurulması (#3 → #4 → #5).

synchronize / opened / reopened / edited
PR olay türleri. İlk üçü CI'ı başlatır; edited başlatmaz.

Testcontainers
Testler için Docker'da gerçek Postgres/Redis açan kütüphane.

Variables
Railway'deki canlı ayarlar; `.env`'in canlı karşılığı.

Wait for CI
Railway ayarı: GitHub kontrolleri yeşil bitmeden yayınlama.

Webhook ("zil")
Bir olay olunca bir servisin diğerine otomatik gönderdiği haber.

Workflow
Actions'ta bir tarif dosyası; bizde tek: `ci.yml` (CI/CD Pipeline).

Bölüm 9 · KOPYA KÂĞIDI

Değişikliği canlıya alma — 10 adım

1. **PR'ı aç.** Claude'un verdiği bağlantı ya da github.com/erdemsidal/collab-board → Pull requests → Open → başlık.
2. **Hedefi kontrol et.** Başlığın altında "... into `main` from ..." yazmalı.
3. **"Not" bölümünü oku.** Önce yapılacak iş (değişken, veritabanı...) varsa şimdi yap.
4. **Railway açık mı?** Servisler durdurulmuşsa: `Postgres` → Deployments → ⋮ → Redeploy; aynısı `Redis`; ikisi de **Online** olana kadar bekle.
5. **En alta in** (End): ✓ All checks have passed ve ✓ no conflicts. Kırmızı ✗ / sarı ● / conflict → **dur**, Claude'a yaz.
6. **Yeşil** Merge pull request (oka değil, düğmenin kendisine).
7. Confirm merge — metinlere dokunma. Mor Merged görününce istersen Delete branch.
8. **Railway'i izle**: `collab-board` → Deployments: Waiting for CI → Initializing → Building → Deploying → ACTIVE. Toplam ~7 dk.
9. **Logları kontrol et**: View logs → `Started` var mı? `level_value` 40000 (ERROR) var mı?
10. **Siteyi dene**: aç, Ctrl+F5, değişikliği gözünle gör. Sorun → Deployments'ta önceki sürüm → ⋮ → Redeploy, ve Claude'a yaz.

Yalnızca ayar değişikliği

PR yok. `collab-board` → Variables → değiştir → üstte Apply N changes → Deploy.

İşin bitince kapatmak

Sıra: `collab-board` → `Postgres` → `Redis`. Her biri: Deployments → ACTIVE → ⋮ → Remove. Veri silinmez.

Asla

Kırmızı ✗ ile birleştirme · `.env`'i ya da bir anahtarı GitHub'a koyma · uygulanmış bir migration dosyasını değiştirme.

Yayın başlamazsa

Ctrl+K → `deploy` → Deploy Latest Commit; sonra GitHub → Settings → Applications → Railway → Configure.